~/python-brasil-2026

$ uv run palestra --tema temporal

Python + Temporal

O core de workflows modernos

Adilson Torres · Engenheiro de Software · DevOps · automação de redes

github.com/AdilsonTorres/python-temporal-core

PYTHON BRASIL 2026



$ cd secao-1 && cat README

Abertura

a dor de todo dev Python que já precisou garantir que algo termine

exemplos e slides

$ whoami # quem fala



Adilson Torres

Engenheiro de Software · DevOps · NetDevOps

8 anos de backend, DevOps e automação de redes

NIC.br · time do IX.br: automação da rede com Temporal, SAGA e FastAPI

Python no dia a dia: scripts e automações, Django, FastAPI, Nornir

Engenharia de Computação · Poli-USP

github.com/AdilsonTorres

linkedin.com/in/adilson-torres

$ whoami --trabalho # onde trabalho

NIC.br e o time do IX.br

nic.br

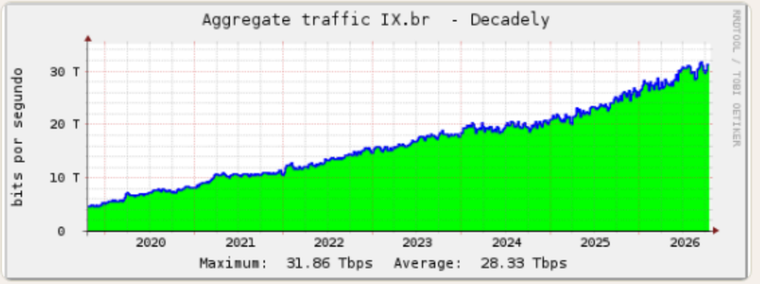
Núcleo de Informação e Coordenação do Ponto BR

Sem fins lucrativos: executa os projetos do CGI.br, o Comitê Gestor da Internet no Brasil.

Registro.br (6 milhões de domínios .br) · CERT.br · Cetic.br · Ceptro.br · Ceweb.br · **IX.br**



Ponto de Troca de Tráfego: as redes se conectam direto, perto do usuário: mais rápido e mais barato. É o maior conjunto de PTTs do mundo, com mais de 3,7 mil redes.



40
PTTs, nos 26 estados

55 Tbit/s
recorde de tráfego (jun/2026)

fontes: nic.br · ix.br (release de 23/09/2026) · gráfico: tráfego médio

$ whoami --talks # por onde passei

Da primeira Python Brasil até aqui



2019	**Python Brasil 2019 · Ribeirão Preto** · participante minha primeira Python Brasil
2019	**Grupy-SP · São Paulo** · palestrante 25 de maio · minha primeira palestra: signals
2025	**PyCon US 2025 · Pittsburgh** · participante Pydantic AI, OpenTelemetry, Python 3.14 e free-threading
2025	**11ª Semana de Capacitação · NIC.br** · co-palestrante Event-Driven Network Automation na Prática
2026	**12ª Semana de Capacitação · NIC.br** · instrutor Temporal (não é sobre chuva): resiliência na automação de redes
2026	**Temporal Replay · San Francisco → PyCon US · Long Beach** · participante maio, nessa ordem · workshops de agentes de IA duráveis e MCP
hoje	**Python Brasil 2026** · palestrante Python + Temporal: o core de workflows modernos

```
$ tail -f /var/log/pedidos.log  # um job de e-commerce, 7 passos em sequência
```

```
03:07 · alerta no celular: o pedido #4821 parou no meio
```

Passo 4 de 7 falhou. E agora?

passo 1	passo 2	passo 3	passo 4	passo 5	passo 6	passo 7
validar pedido	reservar estoque	cobrar cartão	emitir nota fiscal	separar no estoque	avisar cliente	fechar pedido
✅ feito	✅ feito	✅ feito	❌ API fora do ar	⏸️ não rodou	⏸️ não rodou	⏸️ não rodou

O cartão **já foi cobrado**. Rodar tudo de novo cobra duas vezes; não rodar deixa o pedido pela metade.

É o problema de qualquer código com vários passos que precisa chegar ao fim.

$ cat exemplos/e00_problema/fragil.py # o "antes", simplificado

Uma máquina de estados escrita à mão

fragil.py

```python
passo = ler_status_do_banco(pedido_id)
if passo == "validado":
    com_retry("cobrar")
    # o processo morre AQUI...
    salvar_status("cobrado")   # cobra duas vezes?
if passo == "cobrado":
    try: com_retry("reservar_estoque")
    except RuntimeError: com_retry("estornar")  # e se falhar?
```

E ainda falta o cron que acha pedidos "presos", os alertas e o painel de status.

```
$ ls complexidade_acidental/  # o zoológico da complexidade acidental
```

Nada disso é regra de negócio

tenacity
retry dentro de um processo que pode morrer

Celery / RQ
tasks soltas; o estado do processo fica com você

cron
sem histórico, sem retry, sem "rodou ontem?"

tabela de status
máquina de estados à mão

idempotency key
para não repetir o que já foi feito

reconciliação
o script que conserta o que os outros quebraram

$ cd secao-2 && cat README

Conceitos

Durable Execution, histórico + replay e o SDK Python

> durable_execution --explain

Você escreve o caminho feliz.

O Temporal (open source) cuida de retries, estado, timers e recuperação. Se o processo morrer, a execução continua exatamente de onde parou.



$ temporal --version --historia # o que é o temporal

Temporal: Durable Execution open source

de onde veio

2012 **AWS Simple Workflow** (SWF): Maxim Fateev e Samar Abbas no time que lançou

depois Samar cria o **Durable Task Framework** na Microsoft, base do Azure Durable Functions

Uber os dois criam o **Cadence**, open source

2019 fundam a **Temporal**: um fork do Cadence, open source (MIT)

SDKs oficiais

`Go` `Java` `Python` `TypeScript` `.NET` `PHP` `Ruby`

+ Rust em preview · self-hosted ou Temporal Cloud

quem usa

Netflix · Stripe · Snap · OpenAI · NVIDIA · JPMorgan Chase · Datadog

segundo os casos publicados pela Temporal

usos: `pagamentos e pedidos` `deploys e infra` `agentes de IA` `pipelines de dados` `onboarding` `redes`

$ temporal workflow show -w pedido-42 # o modelo mental

Histórico de eventos + replay

```
1 · WorkflowExecutionStarted
2 · ActivityTaskCompleted validar_pedido
3 · ActivityTaskCompleted cobrar → "pag-42"
4 · ActivityTaskScheduled reservar_estoque
× worker morre (kill -9)
↻ novo worker: replay de 1-4
5 · ActivityTaskCompleted reservar_estoque
```

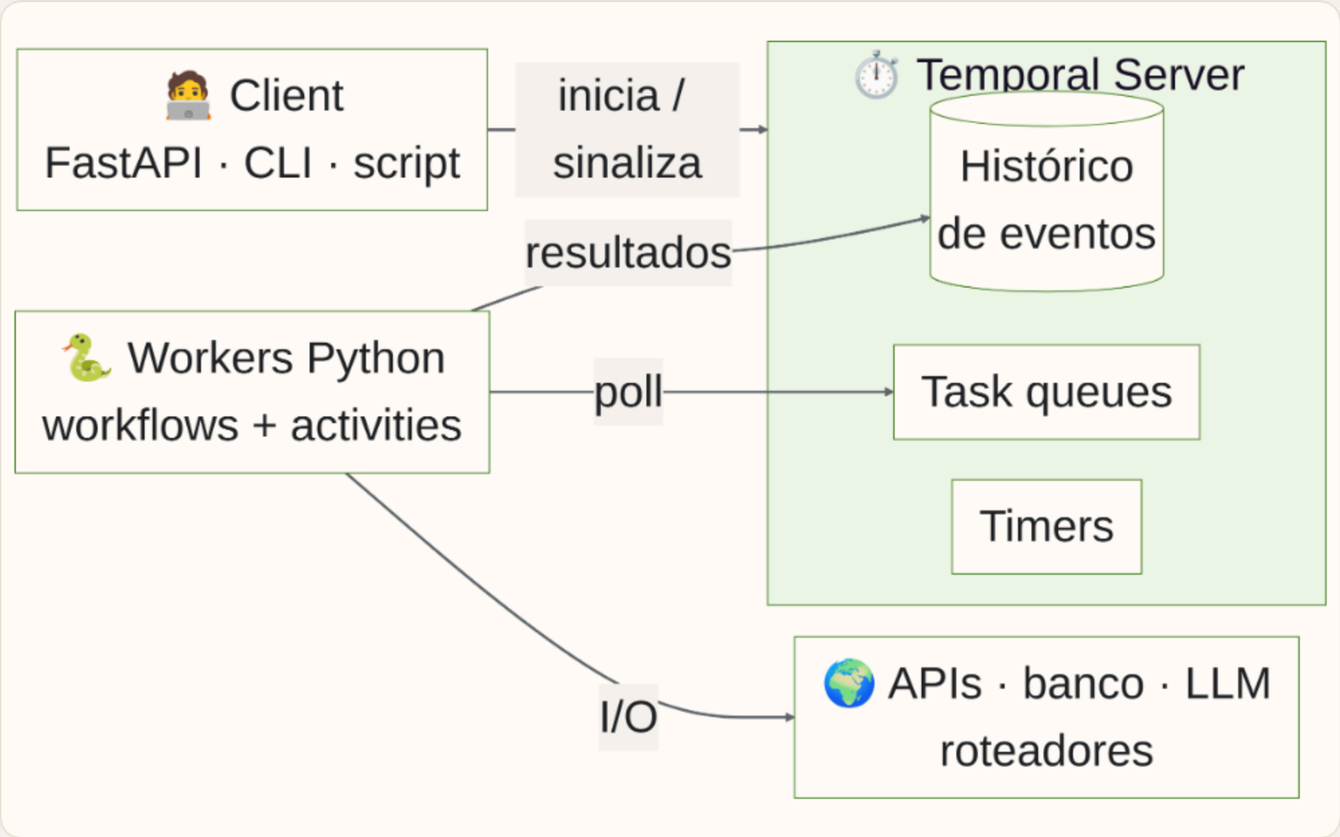
O servidor guarda cada decisão e cada resultado.

O worker (seu processo Python) reexecuta o código do workflow do início; passos já concluídos devolvem o resultado do histórico, sem rodar de novo.

Por isso o workflow precisa ser determinístico.

$ temporal --arquitetura # as peças

Seu código roda nos seus workers



Workflow: a orquestração, determinística

Activity: qualquer I/O, com retry

Worker: processo Python que hospeda os dois

Server: guarda o histórico; não executa seu código

```
$ docker compose ps  # as peças, por dentro
```

O que roda onde

no seu lado (sua infra)

Client
inicia workflows, manda Signal/Update, faz Query

Workers
processos Python com seus Workflows e Activities; buscam tarefas nas filas

⇄

Temporal Service

Frontend
API gRPC (:7233), entrada de tudo

History
histórico de eventos e timers

Matching
task queues

Banco
PostgreSQL, MySQL ou Cassandra (SQLite no dev)

Visibilidade
busca e listas: SQL, ou Elasticsearch/OpenSearch (opcional)

Web UI
a interface que vamos ver já já

CLI / admin-tools
temporal, schema, tdbg

Para desenvolver: `temporal server start-dev` junta tudo num binário, com SQLite e a UI.

$ cat exemplos/e01_pedido/workflows.py # workflow

O caminho feliz, em Python comum

workflows.py

```python
@workflow.defn
class PedidoWorkflow:
    @workflow.run
    async def run(self, pedido: Pedido) -> ResultadoPedido:
        await workflow.execute_activity(
            validar_pedido, pedido, start_to_close_timeout=TIMEOUT)
        pag = await workflow.execute_activity(
            cobrar, pedido, start_to_close_timeout=TIMEOUT)
        # reservar, enviar, notificar... sem tabela de status
```

decorators | asyncio | type hints | Pydantic | retry declarado

$ cat exemplos/e01_pedido/{activities,worker}.py # activity + worker (simplificado)

O I/O mora nas activities

activities.py · worker.py

```python
@activity.defn
async def cobrar(pedido: Pedido) -> str:
    # I/O de verdade: pode falhar e rodar mais de uma vez
    resp = await gateway.cobrar(pedido.valor, chave=pedido.id)
    return resp.id

await Worker(client, task_queue="pedidos",
    workflows=[PedidoWorkflow], activities=[cobrar]).run()
```

Qualquer lib serve: **httpx, SQLAlchemy, boto3**. A regra é uma só: **idempotente** (rodar 2× = rodar 1×).

$ man determinismo # a única regra nova

O workflow precisa ser determinístico

No workflow, não use	Use
`datetime.now()`	`workflow.now()`
`uuid.uuid4()` · `random`	`workflow.uuid7()` · `workflow.random()`
`time.sleep()`	`await workflow.sleep(timedelta(days=30))`
`requests` / `httpx` / banco	uma activity
`threads`	`asyncio.gather` sobre activities

O **sandbox** do SDK Python barra várias dessas chamadas (datetime.now, random…) antes de virarem bug.

$ cd secao-3 && cat README

Na prática

tour da Temporal UI e kill -9 no worker

Web UI

default

local

13 Workflows

Refresh

12 COMPLETED

1 FAILED

18 de out. de 2026, 11:16:12,02 BRT

Start Workflow

- Namespaces
- Workflows
- Standalone Activities
- Schedules
- Agents
- Batch
- Workers
- Nexus
- Archive
- Import
- Docs
- Feedback

2.56.0

All | Failures 0 | Running | Parent | Scheduled | Today | Last 1h | Saved Views 0 / 50

Add Filter

Status	Workflow ID	Run ID	Type
COMPLETED	assinatura-v2-111448	01a14f5e-a705-7e6f-9fb5-4515ca6e3989	AssinaturaWorkflow
COMPLETED	assinatura-v1-111448	01a14f5e-6681-78b4-ac8e-f1cdf03e2f81	AssinaturaWorkflow
COMPLETED	inferencia-1792332939-3	01a14f5e-3f42-7a15-9476-d5bb6f33d955	InferenciaWorkflow
COMPLETED	inferencia-1792332939-2	01a14f5e-3f28-7b84-8fbb-36dff4f79dc2	InferenciaWorkflow
COMPLETED	inferencia-1792332939-1	01a14f5e-3f19-750b-943f-1851932b0837	InferenciaWorkflow
COMPLETED	agente-111448	01a14f5e-10de-71b9-b1f7-15c9bd5a51c0	AgenteOpsWorkflow
COMPLETED	relatorio-diario-2026-10-18T14:15:19Z	01a14f5d-f29d-7e00-9565-a0a291d83423	RelatorioDiarioWorkflow
COMPLETED	lembrete-ana	01a14f5d-e4f9-753e-a38d-fe7f49760ed3	LembreteWorkflow
COMPLETED	deploy-111448	01a14f5d-ca10-70fa-8fdb-2ddf928772eb	DeployComAprovacaoWorkflow
FAILED	viagem-89115e	01a14f5d-ba5a-7f17-9b64-0693c308292d	ReservaViagemWorkflow

100

1–13 of 13

default

local

Back to Workflows

COMPLETED **pedido-tour**

Reset | More Actions

Start	18 de out. de 2026, 11:11:45,8	Run ID	01a14f5a-b065-7a10-b2ce-e90…	History Size	4.23 KB
End	18 de out. de 2026, 11:11:47,5	Workflow Type	PedidoWorkflow	State Transitions	18
Duration	1s 691ms	Task Queue	pedidos	Workflow SDK	Python 1.34.0 Docs

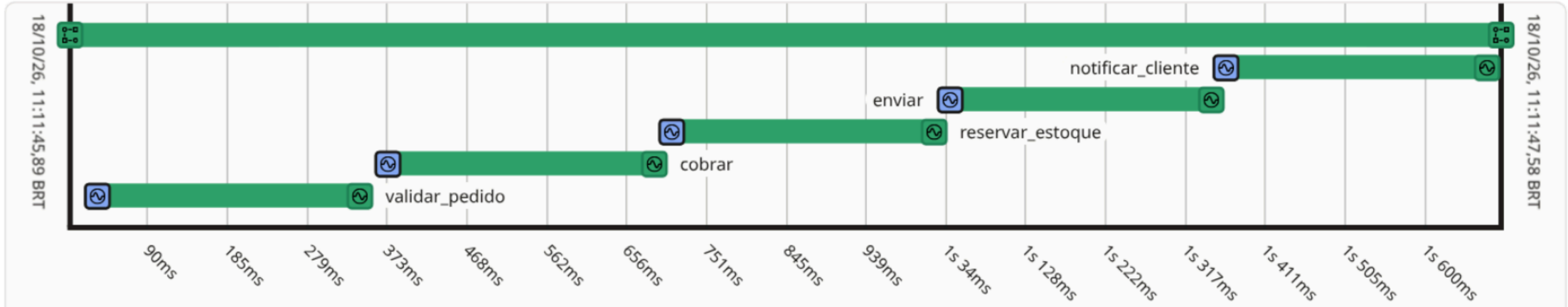
Timeline | Event History 35 | Relationships 0 | Workers 1 | Pending Activities 0 | Call Stack | Queries | User Metadata | Search Attributes | Memo

Input {"id":"tour","cliente":"ana","itens":["teclado","mouse"],"v…

Result {"pedido_id":"tour","pagamento_id":"pag-tour","rastreio":"…

Timeline

Descending | Hide idle time | Filter | Auto Refresh Off | Download



2.56.0

default

local

Back to Workflows

COMPLETED **pedido-tour**

Reset | More Actions

Start	18 de out. de 2026, 11:11:45,8	Run ID	01a14f5a-b065-7a10-b2ce-e90…	History Size	4.23 KB
End	18 de out. de 2026, 11:11:47,5	Workflow Type	PedidoWorkflow	State Transitions	18
Duration	1s 691ms	Task Queue	pedidos	Workflow SDK	Python 1.34.0 Docs

Timeline | Event History 35 | Relationships 0 | Workers 1 | Pending Activities 0 | Call Stack | Queries | User Metadata | Search Attributes | Memo

Input `{"id":"tour","cliente":"ana","itens":["teclado","mouse"],"v…`

Result `{"pedido_id":"tour","pagamento_id":"pag-tour","rastreio":"…`

Event History

All | Compact | JSON

Descending | Filter | Auto Refresh Off | Download

Event ID	Timestamp	Event Type	Details
35	18 de out. de 2026, 11:11:47,58 BRT	Workflow Execution Completed	Result `{"pedido_id":"tour","pagamento_id":"pag-tour","ras…`
34	18 de out. de 2026, 11:11:47,58 BRT	Workflow Task Completed	Identity 1245865@pc-home
33	18 de out. de 2026, 11:11:47,57 BRT	Workflow Task Started	History Size Bytes 4000
32	18 de out. de 2026, 11:11:47,56 BRT	Workflow Task Scheduled	Task Queue Name 1245865@pc-home-0393aa53468b4204aa8527307a2ba171
31	18 de out. de 2026, 11:11:47,56 BRT	Activity Task Completed	Result `{"metadata":{"encoding":"YmluYXJ5L251bGw="}}`
30	18 de out. de 2026, 11:11:47,25 BRT	Activity Task Started	Attempt 1

100 | 1

2.56.0

default

local

Back to Workflows

RUNNING **inferencia-1792332720-5**

Request Cancellation | More Actions

Start	18 de out. de 2026, 11:12:00,9:	Run ID	01a14f5a-eb15-7c9a-a337-9a8…	History Size	676 Bytes
End	-	Workflow Type	InferenciaWorkflow	State Transitions	18
Duration	46s 731ms	Task Queue	fila-gpu	Workflow SDK	Python 1.34.0 Docs

Timeline | Event History 5 | Relationships 0 | Workers 1 | Pending Activities 1 | Call Stack | Queries | User Metadata | Search Attributes | Memo

SCHEDULED **gerar_resposta**

Pause | Update | Reset

Activity ID	1
Attempt	6 / UNLIMITED
Next Retry	18 de out. de 2026, 11:13:01,20 BRT **(14s)**
Last Attempt Completed Time	18 de out. de 2026, 11:12:31,20 BRT
Retry Expiration	No Expiration
Last Heartbeat	18 de out. de 2026, 11:12:31,20 BRT
Last Worker Identity	1247869@pc-home

Last Failure

```
{
  "message": "backend fora do ar: All connection attempts
failed",
  "applicationFailureInfo": {}
}
```

Stack Trace

```
  File "/home/adilson/Projects/github/python-temporal-
core/.venv/lib/python3.14/site-
packages/temporalio/worker/_activity.py", line 351, in
_handle_start_activity_task
    result = await self._execute_activity(
             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
        start, running_activity, task_token, data_converter
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
    )
    ^
```

2.56.0

```
$ make e01-worker & make e01 && kill -9 $WORKER  # na prática
```

kill -9 no worker

bash

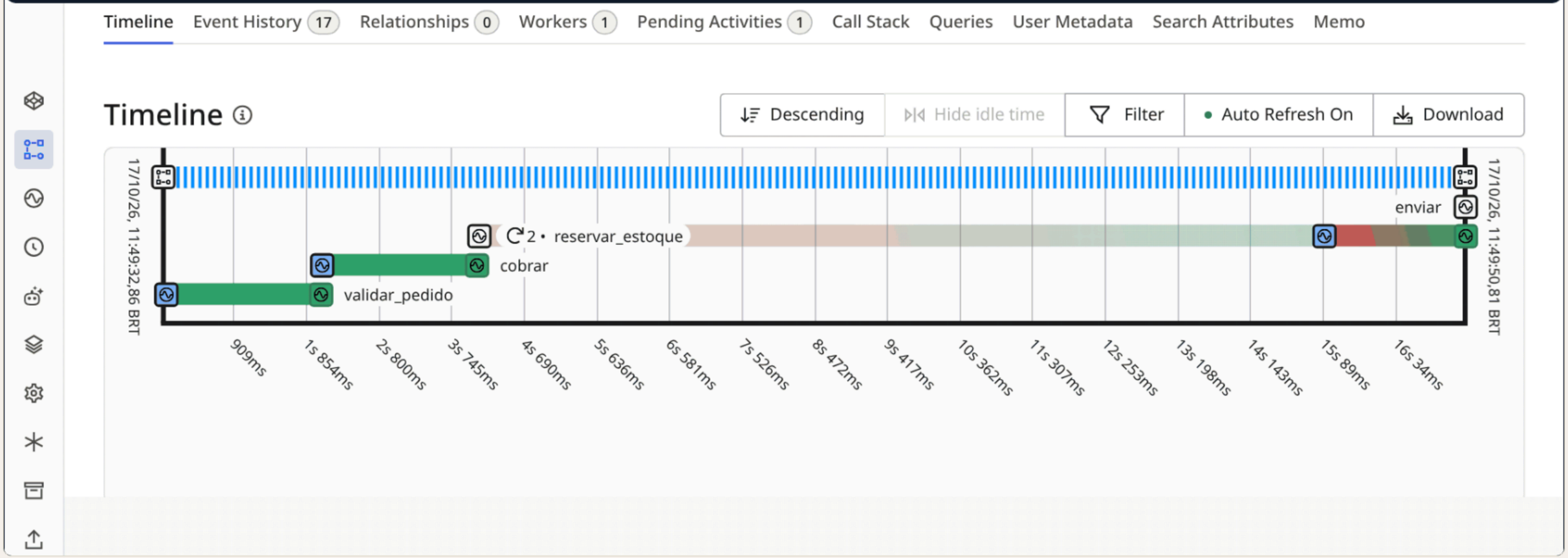
```
# terminal 1  make servidor     # ou: make docker-up
# terminal 2  make e01-worker
# terminal 3  make e01
# no meio do "reservar estoque":
    kill -9 <pid do worker>
# make e01-worker de novo e assiste
```




uv ou Docker · Temporal UI: localhost:8233

e01 · worker morto com kill -9, outro retoma

```
worker | ✅ reservar estoque: pedido demo762 (tentativa 1)
$ kill -9 2289345
💥 worker morto (kill -9)
$ make e01-worker
novo   | 👷 worker de pedidos (pid 2290785)
novo   | ✅ reservar estoque: pedido demo762 (tentativa 2)
novo   | ✅ enviar: pedido demo762 (tentativa 1)
```



```
$ temporal workflow show | tail  # o que aconteceu
```

Ninguém escreveu código de recuperação

0

cobranças duplicadas: "cobrar" veio do histórico no replay

1 passo

reexecutado: a activity que estava rodando quando o worker morreu

Tudo visível

inputs, outputs, tentativas e erros de cada passo na UI

Vale também para deploy, OOM, máquina reiniciada ou rede caída. O mesmo mecanismo cobre todos.

$ cd secao-4 && cat README

Padrões

idempotência, SAGA, aprovação humana e Schedules, com os exercícios rodando

$ cat exemplos/e02_idempotencia/*.py # padrão 1

Workflow ID é chave de idempotência

api.py

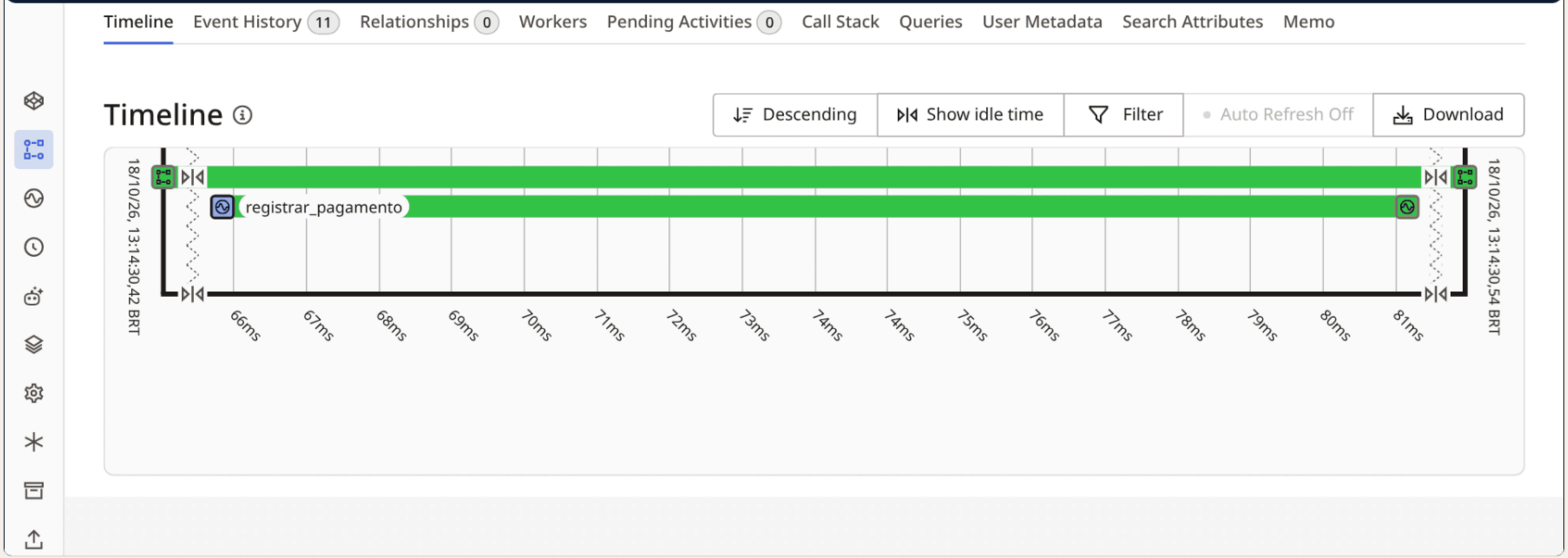
```python
@app.post("/webhooks/pagamento")
async def webhook(evento: EventoPagamento):
    await temporal.start_workflow(
        PagamentoRecebidoWorkflow.run, evento,
        id=f"pagamento-{evento.evento_id}",   # a chave!
        task_queue="pagamentos",
        id_conflict_policy=USE_EXISTING)
```

webhook reenviado 3× → 1 workflow

Deduplicação de graça, sem tabela de eventos processados.

e02 · 3 entregas do webhook, 1 workflow

```
worker| 👷 worker de pagamentos
$ make e02
cli   | 📨 entrega 1: {'workflow_id': 'pagamento-evt_123', 'run_id': '01a14fcb-101b-797b-8dc3-cb73369e6d8b'}
cli   | 📨 entrega 2: {'workflow_id': 'pagamento-evt_123', 'run_id': '01a14fcb-101b-797b-8dc3-cb73369e6d8b'}
cli   | 📨 entrega 3: {'workflow_id': 'pagamento-evt_123', 'run_id': '01a14fcb-101b-797b-8dc3-cb73369e6d8b'}
cli   | ✅ 3 entregas, 1 workflow: pagamento-evt_123
worker| ✅ pagamento registrado: pedido 42
```



$ make e03-falha # padrão 2 · exemplos/e03_saga

SAGA: desfazer em ordem reversa



Carro falhou

erro de negócio: não adianta retentar

Compensa

cancela o hotel, depois o voo

Durável

o rollback continua se o worker cair

$ cat exemplos/e03_saga/workflows.py # padrão 2

É só try/except

workflows.py

```python
compensacoes = []
try:
    voo = await executar(reservar_voo, viagem)
    compensacoes.append((cancelar_voo, voo))
    hotel = await executar(reservar_hotel, viagem)
    compensacoes.append((cancelar_hotel, hotel))
    await executar(reservar_carro, viagem)
except Exception:
    for cancelar, reserva in reversed(compensacoes):
        await executar(cancelar, reserva)
```

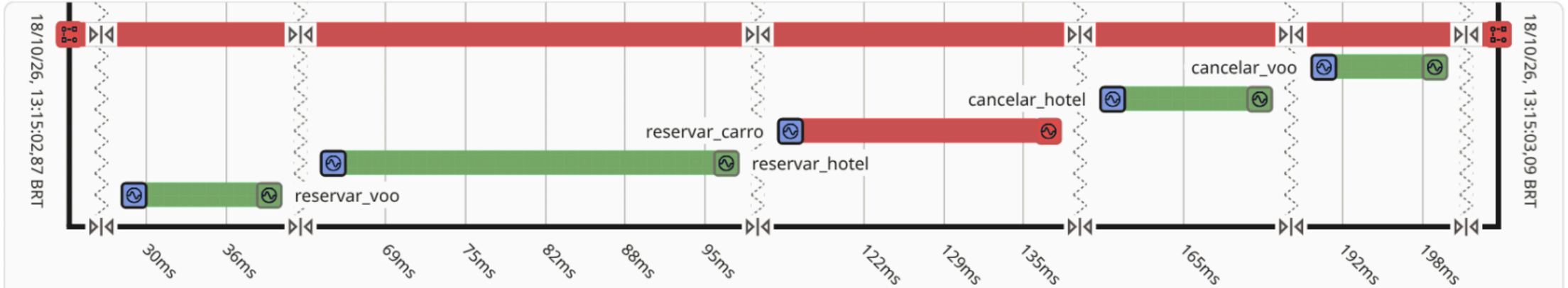
"executar" = workflow.execute_activity com timeout

```
e03 · saga com falha no carro: compensações
$ make e03-falha
worker | ✅ voo reservado para Florianópolis
worker | ✅ hotel reservado em Florianópolis
worker | ❌ carro: sem carros disponíveis
worker | ↩️ compensação: hotel-4a3000 cancelado
worker | ↩️ compensação: voo-4a3000 cancelado
cli    | 💥 viagem falhou e foi desfeita: sem carros disponíveis
```

Timeline | Event History 35 | Relationships 0 | Workers | Pending Activities 0 | Call Stack | Queries | User Metadata | Search Attributes | Memo

Timeline

Descending | Show idle time | Filter | Auto Refresh Off | Download



```
$ make e04-pedir && make e04-aprovar QUEM=bruno  # padrão 3 · exemplos/e04_aprovacao
```

Aprovação humana no meio do fluxo

Ana	`start + Update` Update-with-Start: inicia o workflow e já recebe a resposta
workflow	⏳ `wait_condition` espera até 3 dias
Ana	`Update aprovar(ana)` ❌ validador: não aprove o próprio deploy
Bruno	`Update aprovar(bruno)` ✅ aprovado por bruno
workflow	`activity fazer_deploy` o deploy roda

Update: síncrono, com resposta e validador

Signal: assíncrono, "fire and forget"

Query: lê o estado

Timer de 3 dias: sobrevive a deploys

$ cat exemplos/e04_aprovacao/workflows.py # padrão 3

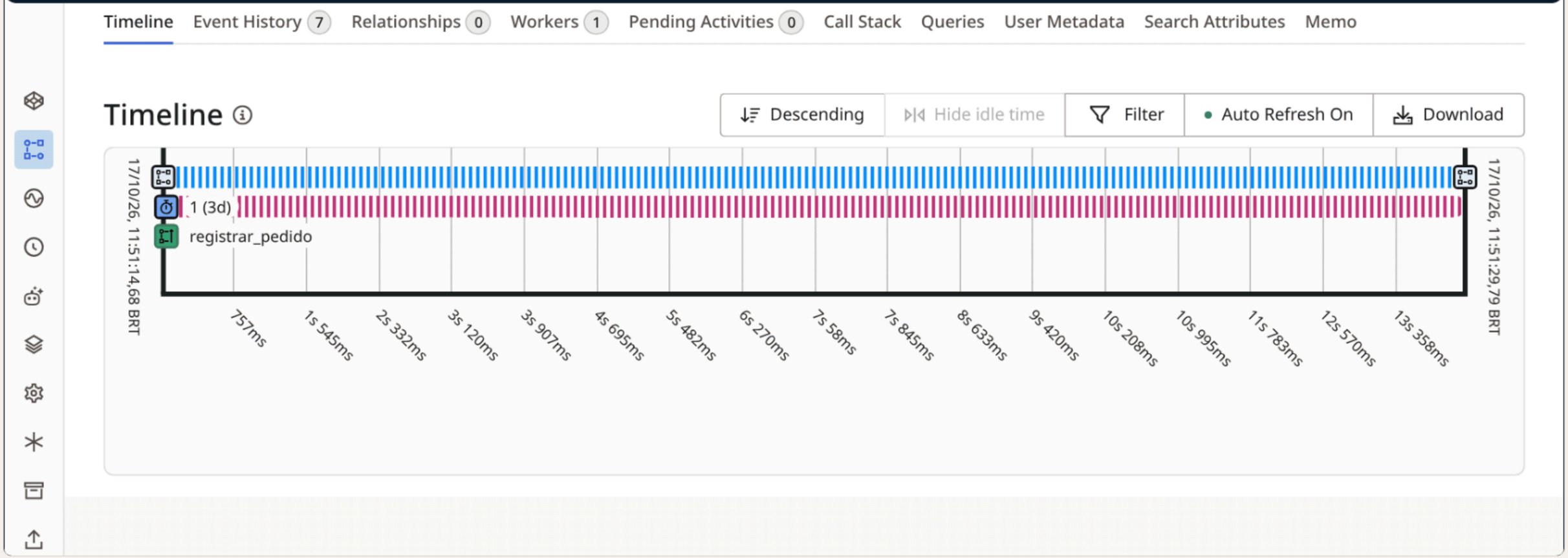
Esperar, aprovar, validar

workflows.py

```python
await workflow.wait_condition(
    lambda: self.aprovador or self.rejeitado_por,
    timeout=timedelta(days=3))

@workflow.update
def aprovar(self, a: Aprovacao) -> str:
    self.aprovador = a.aprovador
@aprovar.validator
def validar(self, a: Aprovacao) -> None:
    if a.aprovador == self.solicitante:
        raise ValueError("quem pediu não pode aprovar o próprio deploy")
```

```
e04 · aprovação humana via Update
$ make e04-pedir ID=deploy-m596
cli    | ⏳ deploy-m596: aguardando aprovação (pedido por ana)
$ make e04-status ID=deploy-m596
cli    | 📋 aguardando aprovação
$ make e04-aprovar ID=deploy-m596 QUEM=ana
cli    | ❌ update rejeitado: ValueError: quem pediu não pode aprovar o próprio deploy
$ make e04-aprovar ID=deploy-m596 QUEM=bruno
```



$ cat exemplos/e05_agendamentos/*.py # padrão 4

Timers duráveis e Schedules

workflows.py

```python
await enviar("bem-vinda(o)!")
await workflow.sleep(timedelta(days=7))   # não segura processo
await enviar("como está sendo sua primeira semana?")

schedule = Schedule(  # o crontab, com histórico
    action=ScheduleActionStartWorkflow(RelatorioDiario.run, ...),
    spec=ScheduleSpec(
        cron_expressions=["0 8 * * 1-5"],  # seg a sex, 8h
        # ou: intervals=[ScheduleIntervalSpec(every=timedelta(hours=1))]
        # ou: calendars=[ScheduleCalendarSpec(hour=[ScheduleRange(8)])]
        time_zone_name="America/Sao_Paulo"),
)
```

pausar · disparar agora · reprocessar dias perdidos · sem execuções sobrepostas · cada execução na UI

make e05 · make e05-agendar

```
e05 · schedule e timer durável
cli    | 📅 schedule 'relatorio-diario' criado
cli    | ▶️ disparado uma vez: veja a aba Schedules na UI
worker | 📊 relatório diário gerado
$ make e05
cli    | ⏳ lembrete em 20s (lembrete-ana)
worker | 📧 para ana: hora da palestra!
cli    | 🔔 lembrete enviado
```

default

local

1 Workflow

Refresh

1 COMPLETED

17 de out. de 2026, 11:52:17,16 BRT

Start Workflow

All | Failures 0 | Running | Parent | Scheduled | Today | Last 1h | *Unsaved view* 0 / 50 | Save as New

WorkflowId starts with "relatorio-diario" | Add Filter | Clear all

Status	Workflow ID	Run ID	Type	Start	E
COMPLETED	relatorio-diario-2026-10-17T14:51:53Z	01a14a59-105a-71d0-aafe-5dfbab8206b2	RelatorioDiarioWorkflow	17 de out. de 2026, 11:51:53,30 BRT	1

100

1–1 of 1

$ cd secao-5 && cat README

Aplicações

agentes de IA duráveis com Pydantic AI e MCP, redes, testes, negócios

$ make e06 # aplicações · agentes de IA

Um agente é um workflow com um LLM no loop



Cada volta do loop é um passo durável: se o worker cair, o agente retoma sem repetir chamadas ao LLM.

funciona com

Pydantic AI · MCP · LangGraph · Strands

$ cat exemplos/e06_agente/agente.py # pydantic ai + temporal

Um agente Pydantic AI, durável

agente.py

```python
agente = Agent(
    modelo, name="agente_ops",
    output_type=[str, DeferredToolRequests],
    capabilities=[TemporalDurability()],   # LLM e tools → activities
)

@agente.tool_plain(requires_approval=True)
async def reiniciar_servico(servico: str) -> str: ...
```

AGENTE_MODELO: vazio = LLM falso (offline) · **ollama:qwen3:8b** = local · **anthropic:**/**openai:** = API · **claude-code** = CLI logado

make e06

$ cat exemplos/e06_agente/workflows.py # pydantic ai + temporal

O workflow espera a aprovação

workflows.py

```python
resultado = await agente.run(pedido)
while isinstance(resultado.output, DeferredToolRequests):
    await workflow.wait_condition(lambda: self.decisao is not None)
    resultado = await agente.run(
        message_history=resultado.all_messages(),
        deferred_tool_results=aprovacoes)
return resultado.output
```

O validador do Update impede o agente de aprovar a si mesmo.

make e06-aprovar QUEM=bruno

```
$ AGENTE_MODELO=claude-code make e06-worker  # um pedido real, LLM de verdade
```

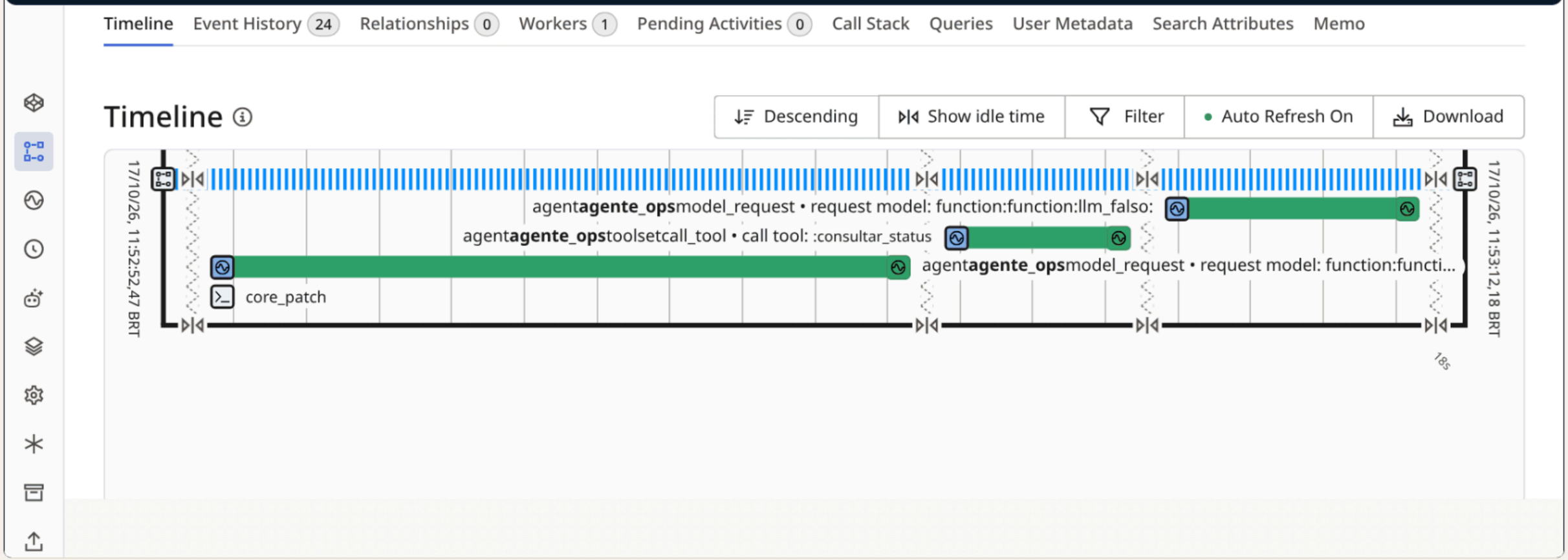
Um pedido real ao Claude

```
e06 · agente com Claude Code

$ AGENTE_MODELO=claude-code make e06-worker &
$ make e06 PEDIDO="O serviço checkout está lento, verifique e resolva"
🤖 agente-claude-11948: agente trabalhando
$ make e06-pendencias
⏳ reiniciar_servico({'servico': 'checkout'}) # pediu aprovação
$ make e06-aprovar QUEM=bruno
✅ aprovado por bruno
🤖 O serviço checkout estava degradado (p99 = 4.2s, 12% de erros 5xx).
   Reiniciei o serviço e a latência foi normalizada.
```

O Claude decidiu reiniciar, mas **só agiu depois do Bruno aprovar**. Cada chamada ao modelo foi uma activity no histórico.

```
e06 · agente de IA com aprovação
$ make e06 AGENTE=agente-m919
cli    | 🤖 agente-m919: agente trabalhando
$ uv run python -m exemplos.e06_agente.cli pendencias agente-m919
cli    | ⏳ ["reiniciar_servico({'servico': 'api-pagamentos'})"]
$ make e06-aprovar AGENTE=agente-m919 QUEM=agente_ops
cli    | ❌ rejeitado: ValueError: o agente não pode aprovar as próprias ações
$ make e06-a
```



```
$ grep -rl temporal ~/projetos  # a mesma base, contextos diferentes
```

Onde já uso Temporal com Python

Automação de redes

webhook do NetBox → FastAPI → workflow → equipamentos, com SAGA e aprovação

Fila de GPU para LLM local

uma inferência por vez; se o Ollama cai, a requisição espera em vez de falhar

Testes automatizados

cada suíte vira activity; só o que falhou roda de novo

Operação e negócio

ingestão para RAG, upgrades com rollback, testes de restauração de backup, lembretes

$ cd secao-6 && cat README

Produção

testes, versionamento e lições aprendidas

$ uv run pytest # produção · testes com pytest



Um timer de 30 dias em milissegundos

test_lembrete.py

```
env = await WorkflowEnvironment.start_time_skipping()
async with Worker(env.client, ...):
    r = await env.client.execute_workflow(
        LembreteWorkflow.run, Lembrete(em=timedelta(days=30)), ...)
assert r == "lembrete enviado"   # roda em ~0,1 s

# no CI: o código novo ainda reproduz os históricos gravados?
await Replayer(workflows=[PedidoWorkflow]).replay_workflow(historico)
```

time-skipping | mock de activity | replay no CI | 38 testes em ~20 s · make test

make test · make replay

$ cat exemplos/e08_versionamento/v2.py # produção

Mudar código com workflows rodando

v2.py

```python
await workflow.execute_activity(cobrar, assinatura, ...)
await workflow.sleep(timedelta(days=30))

if workflow.patched("lembrete-antes-de-renovar"):  # passo novo
    await workflow.execute_activity(enviar_lembrete, assinatura, ...)

return await workflow.execute_activity(renovar, assinatura, ...)
```

Sem **patched()**: execuções que já passaram desse ponto quebram com **NondeterminismError**.

Gate de deploy: replay dos históricos gravados no CI (`make replay`).

patched(): mudança pontual · **Worker Versioning** (Pinned / Auto-Upgrade: cada workflow fica na versão em que começou): o recomendado para produção

`make e08-worker-v1` · `make e08` · `make e08-worker-v2`

$ head -6 licoes.md # produção · o que eu aprendi (1/2)

Seis lições de produção (1/2)

1

Activities idempotentes

Retry e worker que cai depois de executar fazem a activity rodar de novo. Use uma chave de idempotência no serviço externo.

```
gateway.cobrar(valor, chave=pedido.id)
```

2

Payloads pequenos

O histórico guarda todo input e output: até 2 MB por payload e ~50 MB por execução. Guarde o arquivo fora e passe a referência.

```
s3://ingestao/doc-42.pdf # claim check
```

3

Heartbeat em activity longa

Sem heartbeat, o Temporal só descobre que o worker morreu no timeout. Com heartbeat, detecta rápido e a nova tentativa retoma do progresso.

```
activity.heartbeat(i) · heartbeat_timeout=30s
```

$ head -6 licoes.md # produção · o que eu aprendi (2/2)

Seis lições de produção (2/2)

4

Task queues por domínio

Uma fila para rede, outra para NetBox, notificações e GPU. Cada worker escala sozinho e um gargalo não trava os outros.

```
Worker(task_queue="gpu", max_concurrent_activities=1)
```

5

Replayer no CI

Mudar o código de um workflow em andamento quebra o replay. Rode históricos reais no CI antes do deploy e use patched para mudanças.

```
Replayer(workflows=[...]).replay_workflow(historico)
```

6

Observabilidade desde o início

Search attributes para achar execuções por cliente ou pedido, métricas do SDK no Prometheus e alerta para fila sem pollers.

```
upsert_search_attributes · temporal_long_request
```

$ cd secao-7 && cat README

Encerramento

onde o Temporal se encaixa e próximos passos

$ diff celery airflow temporal # onde o temporal se encaixa

Complementa, não substitui

	**Celery / RQ**	tasks curtas · Temporal: **estado durável do processo inteiro**
	**Airflow / Dagster / Prefect**	DAGs de dados · Temporal: **processos interativos e longos**
	**n8n**	low-code · Temporal: **lógica testável e revisável em PR**
	cron	script pontual · Temporal: **histórico, retry, backfill**

"Ele não substitui sua fila nem seu pipeline de dados; substitui o código que você escreve para sobreviver às falhas entre eles."

```
$ which temporal || echo 'cron basta'  # e quando não usar
```

Nem tudo precisa de Durable Execution

Situação	Use no lugar	Por quê
Script de segundos, idempotente	`cron / systemd timer`	nada para lembrar entre execuções
Request/response em ms	`FastAPI direto`	persistir cada passo adiciona latência
Fila de tasks curtas e independentes	`Celery / RQ / Dramatiq`	sem estado entre as tasks
Pipeline de dados em lote	`Airflow / Dagster / Prefect`	agendamento de DAGs, lineage, conectores
Streaming de alto volume	`Kafka / Flink`	eventos por segundo, janelas, agregação
Tudo na AWS, fluxo simples	`Step Functions`	serverless, zero operação

E o custo de operar: self-hosted pede banco, upgrades e backup; o Temporal Cloud cobra por ação. Para duas automações simples, não compensa.

$ git log --since=2026-01 --oneline # encerramento · o que há de novo em 2026

Novidades recentes do ecossistema

Server 1.32 (set/2026)

Standalone Activities GA, Poller Autoscaling, Priority & Fairness

SDK Python 1.32–1.34

workflow.uuid7(), logging JSON, conversor Pydantic mais rápido, Standalone Activities

Agentes

Pydantic AI TemporalDurability, OpenAI Agents SDK, Strands, LangGraph, activity_as_tool

Worker Versioning

Pinned / Auto-Upgrade com ramping, automatizado via GitHub Actions

Fontes: releases temporalio/temporal e sdk-python · blog temporal.io (set/2026)

```
$ git log --grep=fix --oneline  # encerramento · erros que eu cometi
```

Para você não repetir

O erro	O que aprendi
Versionamento PINNED esquecido (workflows presos ao worker antigo): lembretes pararam por 3 h, em silêncio	replay no CI + drenar workers antigos
Workflow "eterno" bateu o limite de updates por execução	continue_as_new quando o servidor sugerir
Heartbeat só no fim de cada rodada	heartbeat contínuo numa task em paralelo
Documentos inteiros no histórico	claim check: guarde fora, passe a referência
Sync de schedules apagou o schedule de outro projeto	dono explícito para cada schedule

$ temporal operator cluster health # encerramento · operar o temporal

Self-hosted sem sustos

Banco

Postgres para histórico e Advanced Visibility: dispensa Elasticsearch

Upgrades

backup → migração de schema → verificação; e um restore drill agendado

Observabilidade

métricas do SDK no Prometheus/Grafana, search attributes para buscar na UI

Canários

DescribeTaskQueue: alerta se uma fila ficar sem pollers

$ temporal --help # encerramento · próximos conceitos

Quando o workflow cresce

Recurso	Quando usar
Child workflow	fan-out com histórico próprio por item; isolar falhas
Continue-as-new	workflows "eternos" (loops, sessões longas): recomeça com histórico limpo
Nexus	chamar operações de outro time ou namespace com contrato
Local activity	passos curtíssimos, sem ida ao servidor por tarefa
Payload codec	criptografar ou comprimir dados no histórico

> cat levar-para-casa.txt # para levar para casa

1. Escreva só o caminho feliz.
2. Workflow determinístico, activity idempotente.
3. É Python comum: asyncio, Pydantic, pytest.
4. A mesma base serve de redes a agentes de IA.

a stack do repo 

make setup · make servidor · make e01-worker · make e01

```
$ exit 0  # obrigado!
```

Perguntas?

Exemplos, testes e roteiro das demos:

`github.com/AdilsonTorres/python-temporal-core`

docs.temporal.io/develop/python

Adilson Torres · linkedin.com/in/adilson-torres



$ cd secao-8 && cat README

Extras: benchmark primeiro, depois exemplos e bastidores

(fora da palestra principal)

```
$ make e09  # extra · exemplos/e09_benchmark_python
```

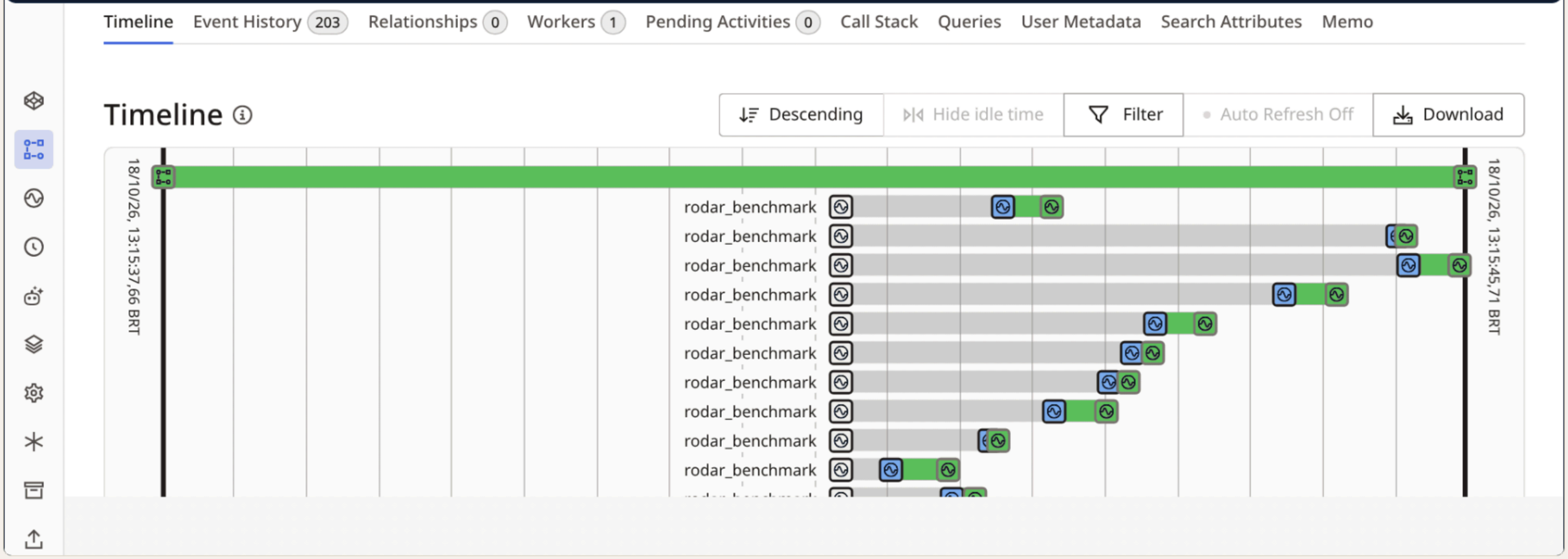
Um workflow que mede 3.13 × 3.14 × 3.15

benchmark	3.13.16	3.14.8	3.15.0
nbody	69,6 ms	64,6 ms · 1,08×	63,6 ms · 1,09×
fib	23,6 ms	21,4 ms · 1,10×	21,1 ms · 1,12×
json	60,8 ms	59,6 ms · 1,02×	54,2 ms · 1,12×
regex	3,4 ms	3,7 ms · 0,92×	3,6 ms · 0,94×
comprehension	76,9 ms	67,7 ms · 1,14×	73,1 ms · 1,05×
média geom.	**1,00×**	**1,05×**	**1,06×**

Uma activity por (versão × benchmark) com **uv run --python**; **max_concurrent_activities=1** para medir de forma justa. Se o worker cair, o que já foi medido não roda de novo.

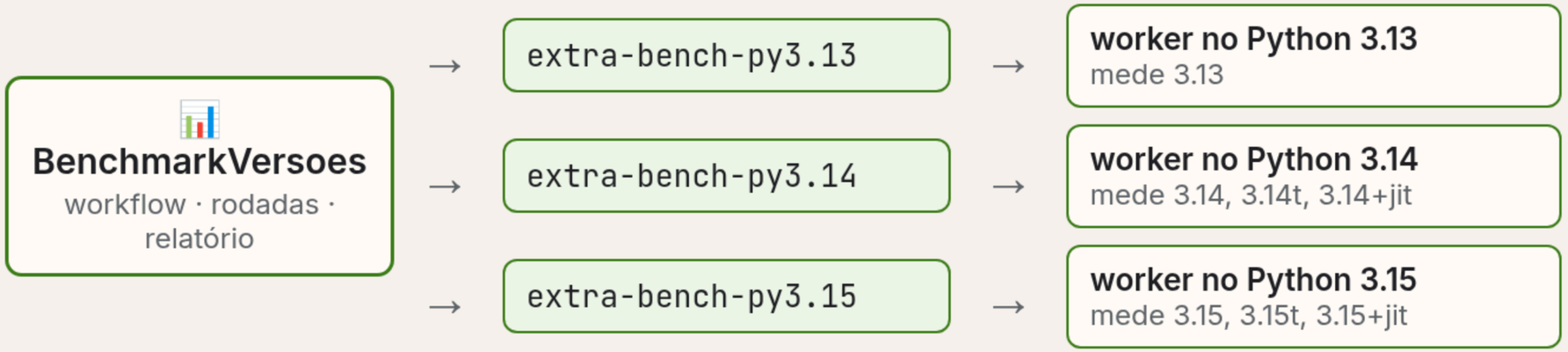
Mediana de 30 medições por célula (10 rodadas × 3); abaixo de ~5% é ruído

```
e09 · benchmark entre versões do Python
cli   nbody              78.0ms ±12% 1.00x    64.2ms ±17% 1.22x    62.7ms ± 9% 1.24x
cli   fib                22.8ms ± 2% 1.00x    23.2ms ±15% 0.98x    20.4ms ± 4% 1.12x
cli   json               57.2ms ± 1% 1.00x    65.2ms ±13% 0.88x    59.1ms ±11% 0.97x
cli   regex               4.3ms ±27% 1.00x     3.6ms ± 3% 1.19x     3.5ms ± 2% 1.23x
cli   comprehension      81.9ms ±13% 1.00x    69.6ms ± 9% 1.18x    67.1ms ± 4% 1.22x
cli   média geom.                   1.00x                 1.08x                 1.15x
cli   mediana ± variação · speedup contra py3.13 (>1 = mais rápido)
```



```
$ make x06-medidores && make x06  # extra (repo privado) · benchmark
```

Como o Temporal roda o benchmark



Roteamento
cada medição vai para a fila da versão certa

Retry + heartbeat
worker caiu? o que mediu fica; o resto é retentado

Rodadas justas
ordem embaralhada por rodada (workflow.random)

Ao vivo
Query progresso/parcial · Signal parar · Update mais_rodadas

$ cat extras/x06_bench_versoes/workflows.py # extra (repo privado) · a estrutura

O benchmark é um workflow comum

workflows.py (resumido)

```python
@workflow.defn
class BenchmarkVersoesWorkflow:
    @workflow.run
    async def run(self, entrada: Entrada) -> Resultado:
        alvos = [alvo_de(v) for v in entrada.versoes + entrada.variantes]
        await asyncio.gather(*[self._ambiente(a) for a in alvos]) # GIL? JIT?
        while self.rodada < self.rodadas and not self.parado:
            pares = [(a, c) for c in cenarios for a in alvos]
            workflow.random().shuffle(pares) # ordem justa a cada rodada
            await asyncio.gather(*[self._medir(a, c) for a, c in pares])
        return Resultado(markdown(self.medicoes))

    async def _medir(self, alvo, cenario):
        return await workflow.execute_activity("medir", Pedido(alvo, cenario),
            task_queue=alvo.fila, # a fila do worker daquela versão
            heartbeat_timeout=timedelta(seconds=30), retry_policy=RETRY)
```

x06 · benchmark 3.13 × 3.14 × 3.15 orquestrado pelo Temporal

```
cli   | async_tarefas | 1.11x | 0.71x | 0.97x | 1.04x |
cli   | json | 0.91x | 0.88x | 1.00x | 0.98x |
cli   | threads_cpu | 3.24x | 2.59x | 1.11x | 1.00x |
cli   | fib | 1.02x | 1.00x | 1.04x | 0.98x |
cli   `Nx` = quantas vezes mais rápido que a referência (>1 é melhor). `±` = variação > 5%.
cli   📄 /home/adilson/Projects/work/pythonbrasil2026/material/exemplos-extras/resultados/bench-versoes-1792248984.md
```

Timeline | Event History 374 | Relationships 0 | Workers 1 | Pending Activities 1 | Call Stack | Queries | User Metadata | Search Attributes | Memo

Timeline

Descending | Hide idle time | Filter | Auto Refresh On | Download

17/10/26, 11:56:24,80 BRT

17/10/26, 11:57:27,68 BRT

salvar_relatorio
medir • 3.15t · json · rodada 2
medir • 3.15 · json · rodada 2
medir • 3.15+jit · threads_cpu · rodada 2
medir • 3.15 · async_tarefas · rodada 2
medir • 3.14+jit · async_tarefas · rodada 2
medir • 3.14+jit · json · rodada 2
medir • 3.13 · threads_cpu · rodada 2
medir • 3.15 · threads_cpu · rodada 2
medir • 3.14 · threads_cpu · rodada 2

default

local

Back to Workflows

RUNNING **bench-versoes-1792249285**

Request Cancellation | More Actions

Start	17 de out. de 2026, 12:01:25,7	Run ID	01a14a61-cc9f-72fa-8881-e3e…	History Size	30.69 KB
End	-	Workflow Type	BenchmarkVersoesWorkflow	State Transitions	157
Duration	26s 136ms	Task Queue	extra-bench	Workflow SDK	Python 1.34.0 Docs

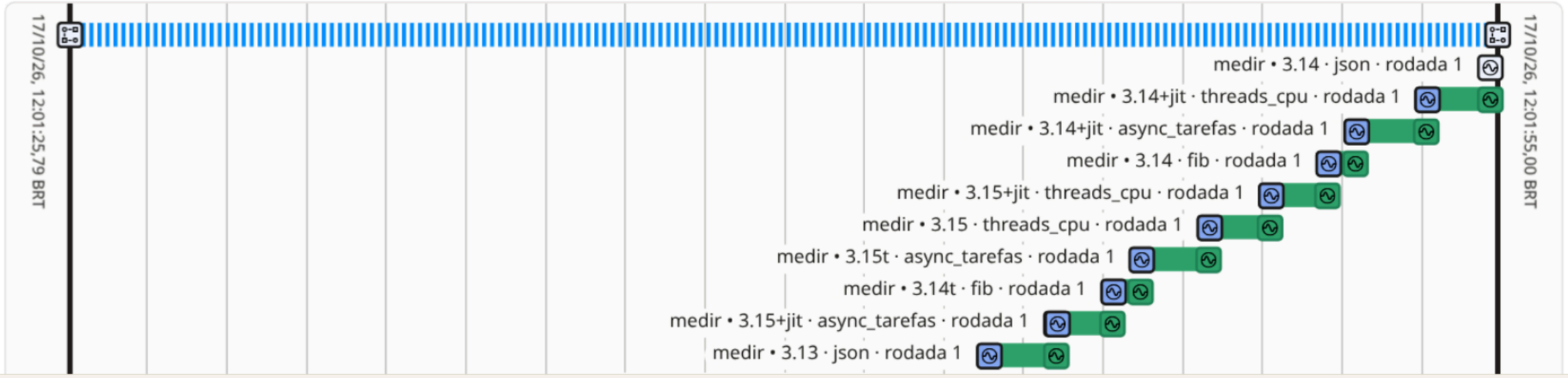
Timeline | Event History 188 | Relationships 0 | Workers 2 | Pending Activities 1 | Call Stack | Queries | User Metadata | Search Attributes | Memo

Input {"versoes":["3.13","3.14","3.15"],"variantes":["3.14t","3.1…

Result Results will appear upon completion.

Timeline

Descending | Hide idle time | Filter | Auto Refresh On | Download



2.56.0

default

local

Back to Workflows

RUNNING **bench-versoes-1792249285**

Request Cancellation | More Actions

Start	17 de out. de 2026, 12:01:25,7	Run ID	01a14a61-cc9f-72fa-8881-e3e…	History Size	33.7 KB
End	-	Workflow Type	BenchmarkVersoesWorkflow	State Transitions	172
Duration	29s 690ms	Task Queue	extra-bench	Workflow SDK	Python 1.34.0 Docs

Timeline | Event History 206 | Relationships 0 | Workers 2 | Pending Activities 1 | Call Stack | Queries | User Metadata | Search Attributes | Memo

STARTED **medir**

Pause | Update | Reset

Activity ID	35
Attempt	1 / 5 4 remaining
Last Heartbeat	17 de out. de 2026, 12:01:55,31 BRT
Last Started Time	17 de out. de 2026, 12:01:55,30 BRT
Last Worker Identity	2353624@pc-home

2.56.0

default

local

Back to Workflows

RUNNING **bench-versoes-1792249285**

Request Cancellation | More Actions

Start	17 de out. de 2026, 12:01:25,7	Run ID	01a14a61-cc9f-72fa-8881-e3e…	History Size	35.69 KB
End	-	Workflow Type	BenchmarkVersoesWorkflow	State Transitions	182
Duration	31s 800ms	Task Queue	extra-bench	Workflow SDK	Python 1.34.0 Docs

Timeline | Event History 218 | Relationships 0 | Workers 2 | Pending Activities 1 | Call Stack | Queries | User Metadata | Search Attributes | Memo

JSON Formatting

Query Type *

parcial

Query Arg

Single JSON payload supported.

Refresh Query

```
# Benchmark: Python 3.13 × 3.14 × 3.15

Interpretadores: **3.15t** = 3.15.0 sem GIL · **3.15** = 3.15.0 ·
**3.14** = 3.14.8 · **3.15+jit** = 3.15.0 +JIT · **3.14t** = 3.14.8
sem GIL · **3.13** = 3.13.16 · **3.14+jit** = 3.14.8 +JIT
Rodadas: 1 · repetições por medição: 2 · paralelismo: 1 · medições:
32

## bibliotecas

| cenário | 3.13 | 3.14 | 3.15 | vencedor |
|---|---:|---:|---:|---|
| json | 413 ms | 455 ms (0.91x) | 387 ms (1.07x) | 3.15 (1.17x) |

## cpu

| cenário | 3.13 | 3.14 | 3.15 | vencedor |
|---|---:|---:|---:|---|
| fib | 160 ms | 148 ms (1.08x) | 140 ms (1.15x) | 3.15 (1.15x) |

## asyncio
```

2.56.0

default

local

15 Workers

Refresh

15 RUNNING

17 de out. de 2026, 12:02:34,84 BRT

Create Worker Deployment

Instances Deployments

All Running Saved Views 0 / 50

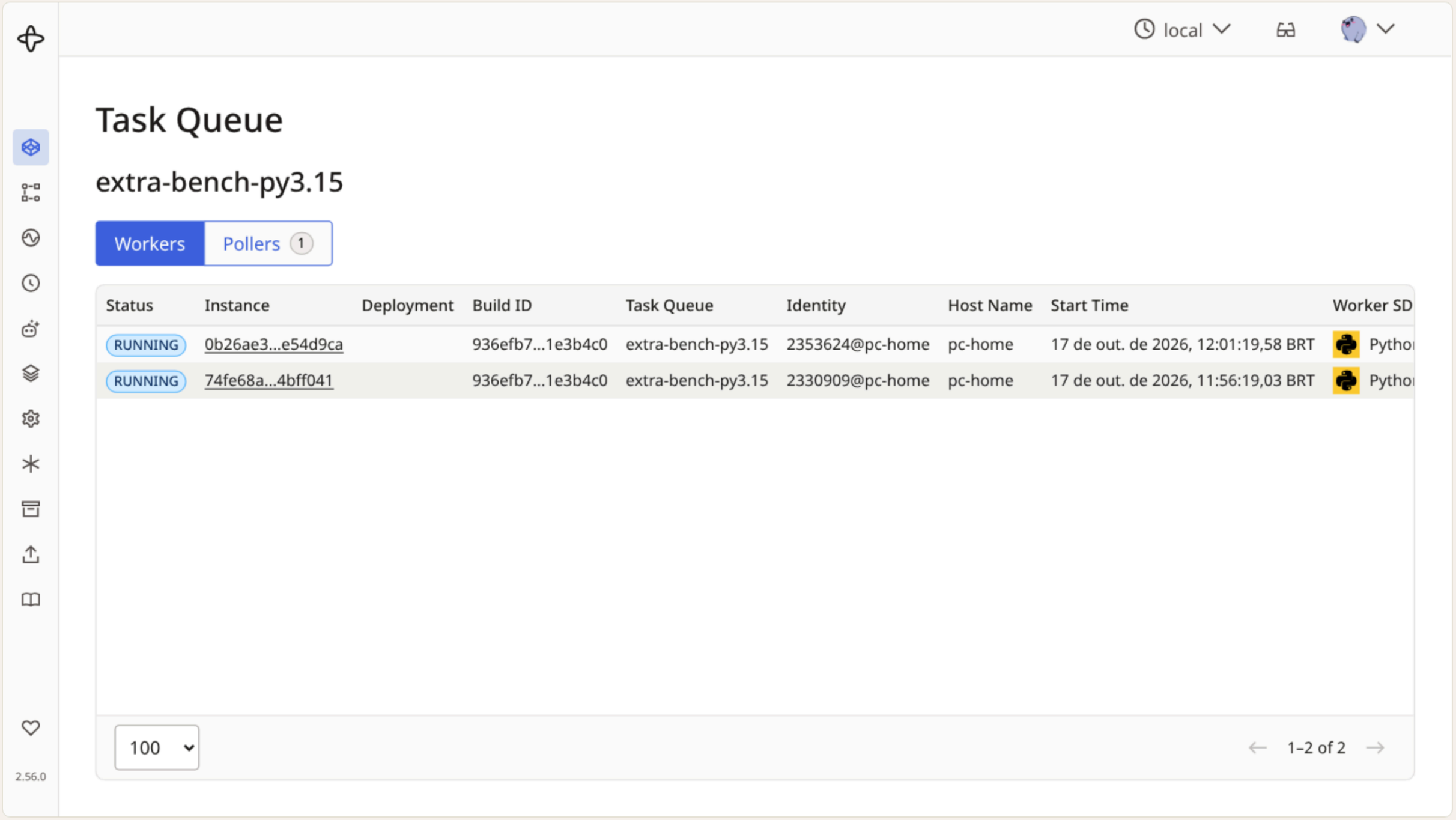
Add Filter

Status	Instance	Deployment	Build ID	Task Queue	Identity	Host Name	Start Time	Worker SD
RUNNING	0b26ae3...e54d9ca		936efb7...1e3b4c0	extra-bench-py3.15	2353624@pc-home	pc-home	17 de out. de 2026, 12:01:19,58 BRT	Pythc
RUNNING	0f34c0a...4308448		a462805...4a14077	extra-bench-py3.14	2330869@pc-home	pc-home	17 de out. de 2026, 11:56:18,54 BRT	Pythc
RUNNING	1c5a33a...55f81a8		ecb3846...0de6513	extra-telegram	2343411@pc-home	pc-home	17 de out. de 2026, 11:58:56,56 BRT	Pythc
RUNNING	39cf6f2...803cdf3		d7313f7...1196a54	extra-bench-py3.13	2353625@pc-home	pc-home	17 de out. de 2026, 12:01:19,58 BRT	Pythc
RUNNING	464e8d7...accd12f		b26fdf9...7d9df61	extra-agente-cli	2341362@pc-home	pc-home	17 de out. de 2026, 11:58:26,03 BRT	Pythc
RUNNING	68e3a0a...a68592f		a462805...4a14077	extra-bench-py3.14	2353606@pc-home	pc-home	17 de out. de 2026, 12:01:19,41 BRT	Pythc
RUNNING	6b4d5c0...4d9d347		362f373...88d5286	agendamentos	2346308@pc-home	pc-home	17 de out. de 2026, 11:59:38,27 BRT	Pythc
RUNNING	7389f9b...b5500d8		91f04b8...aaf1d03	extra-bench	2330973@pc-home	pc-home	17 de out. de 2026, 11:56:19,81 BRT	Pythc
RUNNING	74fe68a...4bff041		936efb7...1e3b4c0	extra-bench-py3.15	2330909@pc-home	pc-home	17 de out. de 2026, 11:56:19,03 BRT	Pythc
RUNNING	854cd6f...69119f6		d7313f7...1196a54	extra-bench-py3.13	2330910@pc-home	pc-home	17 de out. de 2026, 11:56:19,03 BRT	Pythc

100

1–15 of 15

2.56.0


local
Task Queue
extra-bench-py3.15
Workers
Pollers 1
Status Instance Deployment Build ID Task Queue Identity Host Name Start Time Worker SD
RUNNING 0b26ae3...e54d9ca 936efb7...1e3b4c0 extra-bench-py3.15 2353624@pc-home pc-home 17 de out. de 2026, 12:01:19,58 BRT Pytho
RUNNING 74fe68a...4bff041 936efb7...1e3b4c0 extra-bench-py3.15 2330909@pc-home pc-home 17 de out. de 2026, 11:56:19,03 BRT Pytho
100
1–2 of 2
2.56.0

Na média, 3.15 é 10% mais rápido que 3.13

média geométrica de 20 cenários · barras = vezes mais rápido que 3.13

empates: 4 cenários



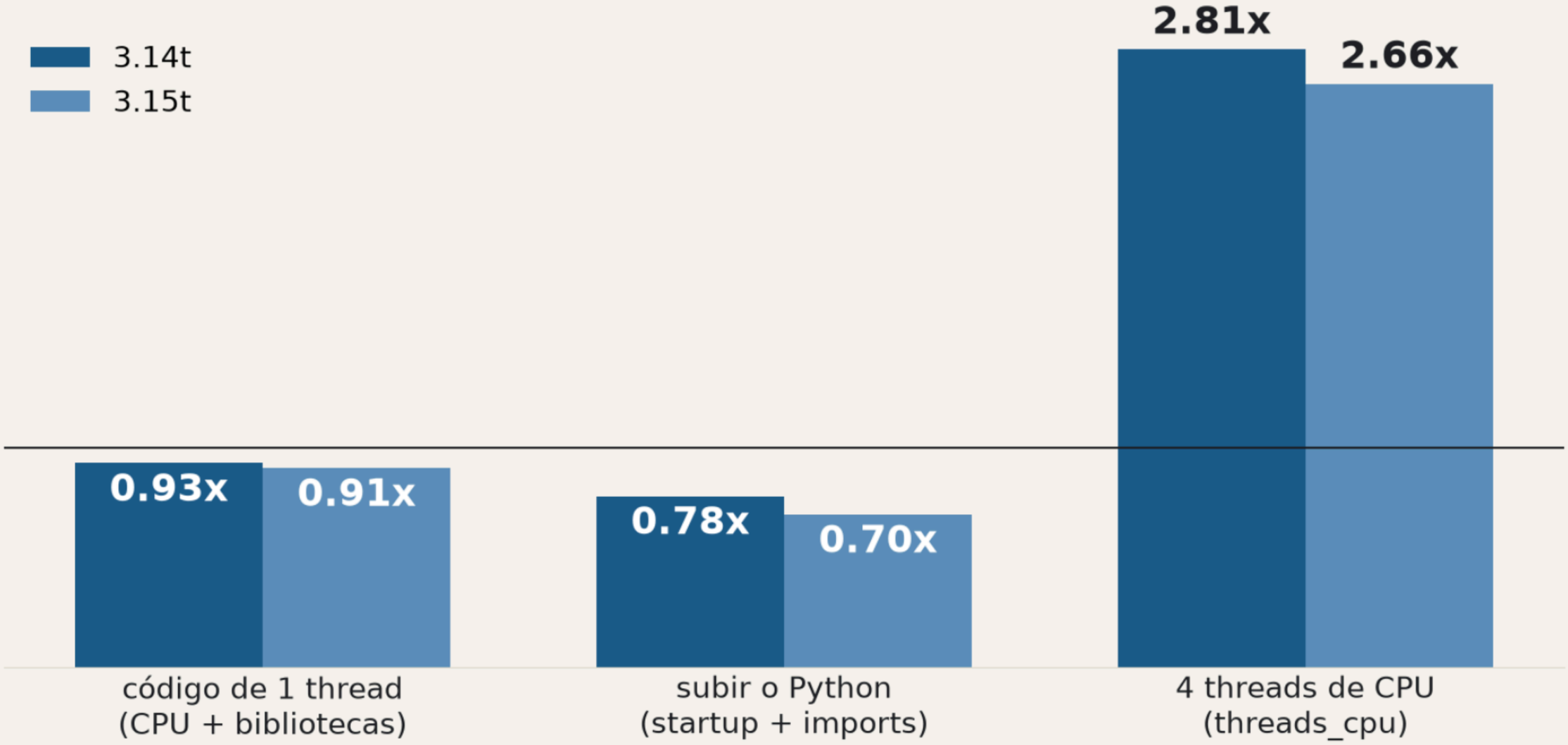
Quem ganha em cada cenário

★ = a mais rápida · número = vezes mais lenta que a melhor · "=" empate (≤3%)

	3.13	3.14	3.15
chamadas · cpu	**1.18**	★	**1.11**
comprehension · cpu	**1.05**	★	**1.04**
exceções · cpu	**1.19**	★	**1.04**
fib · cpu	**1.16**	**1.04**	★
geradores · cpu	**1.28**	**1.10**	★
nbody · cpu	**1.07**	**1.02**	★
objetos · cpu	**1.25**	★	**1.10**
spectral_norm · cpu	**1.39**	**1.16**	★
strings · cpu	=	=	=
async_fila · asyncio	=	=	=
async_tarefas · asyncio	**1.31**	★	**1.02**
decimal · bibliotecas	**1.12**	**1.15**	★
hashlib · bibliotecas	=	=	=
json · bibliotecas	**1.10**	**1.09**	★
regex · bibliotecas	**1.07**	★	★
sort · bibliotecas	**1.03**	★	**1.03**
threads_cpu · threads	**1.18**	**1.06**	★
threads_io · threads	=	=	=
imports · inicialização	**1.01**	**1.04**	★
startup · inicialização	**1.13**	**1.08**	★

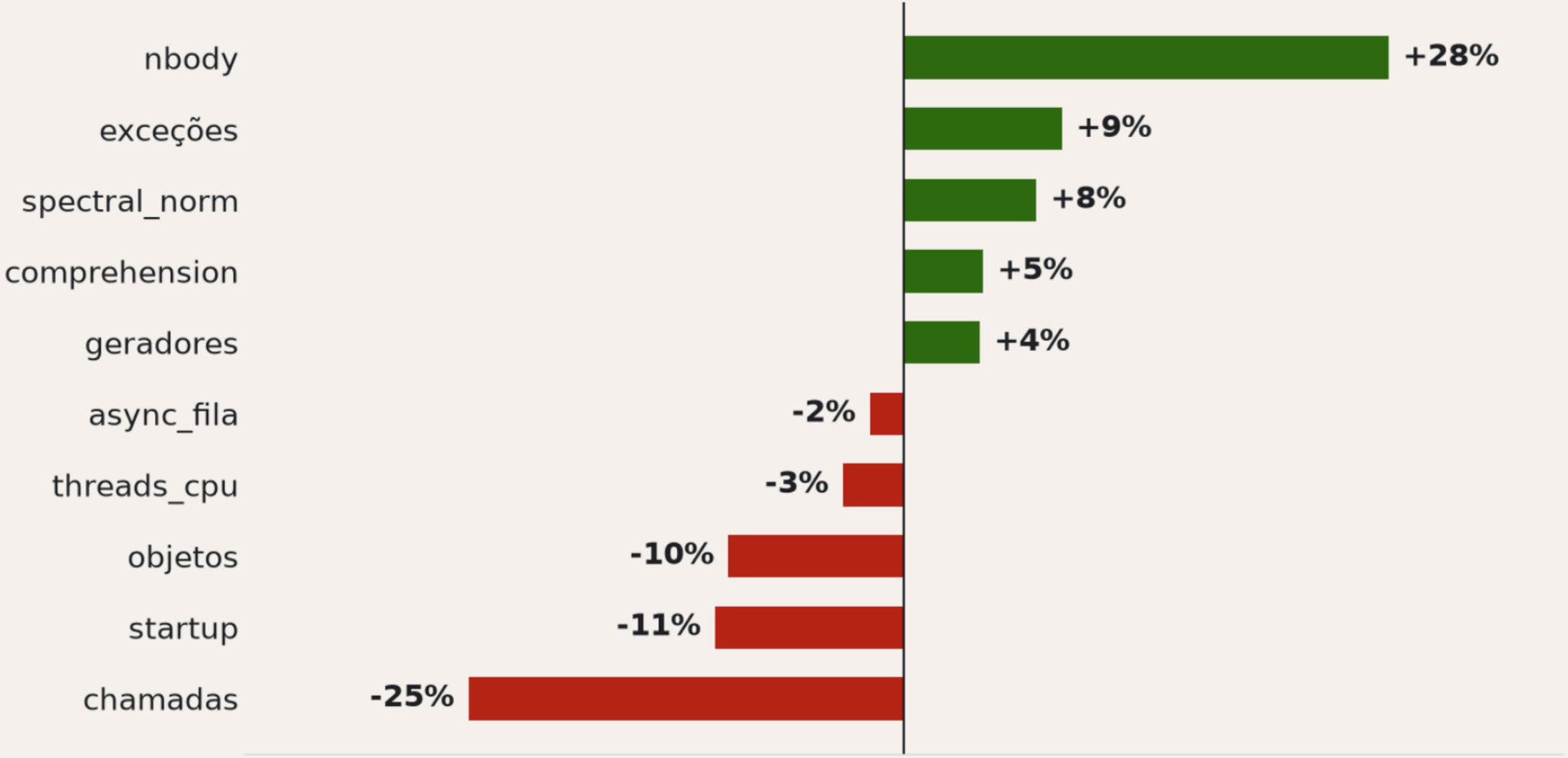
Sem GIL: perde sozinho, ganha com threads

3.14t e 3.15t contra a mesma versão com GIL · 1.0 = igual · >1 = mais rápido



JIT da 3.15 (experimental): ganha em uns, perde em outro

os 5 cenários que mais ganham e os 5 que mais perdem · o resto fica igual



$ cat resultados/bench-versoes-*.md # extra (repo privado) · o que o benchmark mostrou

Onde cada versão ganha

3.15 ganha em CPU e bibliotecas

spectral_norm 1.39x, geradores 1.28x, decimal 1.12x, json 1.10x

3.14 ganha em objetos e asyncio

objetos 1.25x, exceções 1.19x, muitas tasks asyncio 1.31x

Empates

hashlib, strings, threads de I/O e fila asyncio: o trabalho é C ou espera

Sem GIL

4 threads de CPU 2.7-2.8x mais rápido; uma thread só e startup ficam mais lentos

JIT

experimental: nbody 1.28x na 3.15; chamadas pequenas 25% (3.15) a 34% (3.14) mais lentas

Média

3.14 = 1.08x e 3.15 = 1.10x contra a 3.13 (suíte do x06: 10 rodadas, 1.400 medições)

make x06 ARGS="--rodadas 10" · uv run --with matplotlib python -m extras.x06_bench_versoes.graficos

$ cat exemplos/e07_fila_gpu/workflows.py # extra

Fila de GPU para LLM local (1/2)

workflows.py

```
ESPERAR_BACKEND = RetryPolicy(
  initial_interval=timedelta(seconds=2),
  maximum_interval=timedelta(seconds=30),
  maximum_attempts=0)  # sem limite

await workflow.execute_activity(
  gerar_resposta, prompt,
  schedule_to_close_timeout=timedelta(hours=1),
  heartbeat_timeout=timedelta(seconds=30),
  retry_policy=ESPERAR_BACKEND)
```

$ cat exemplos/e07_fila_gpu/worker.py # extra

Fila de GPU para LLM local (2/2)

worker.py

```
Worker(..., max_concurrent_activities=1)
```

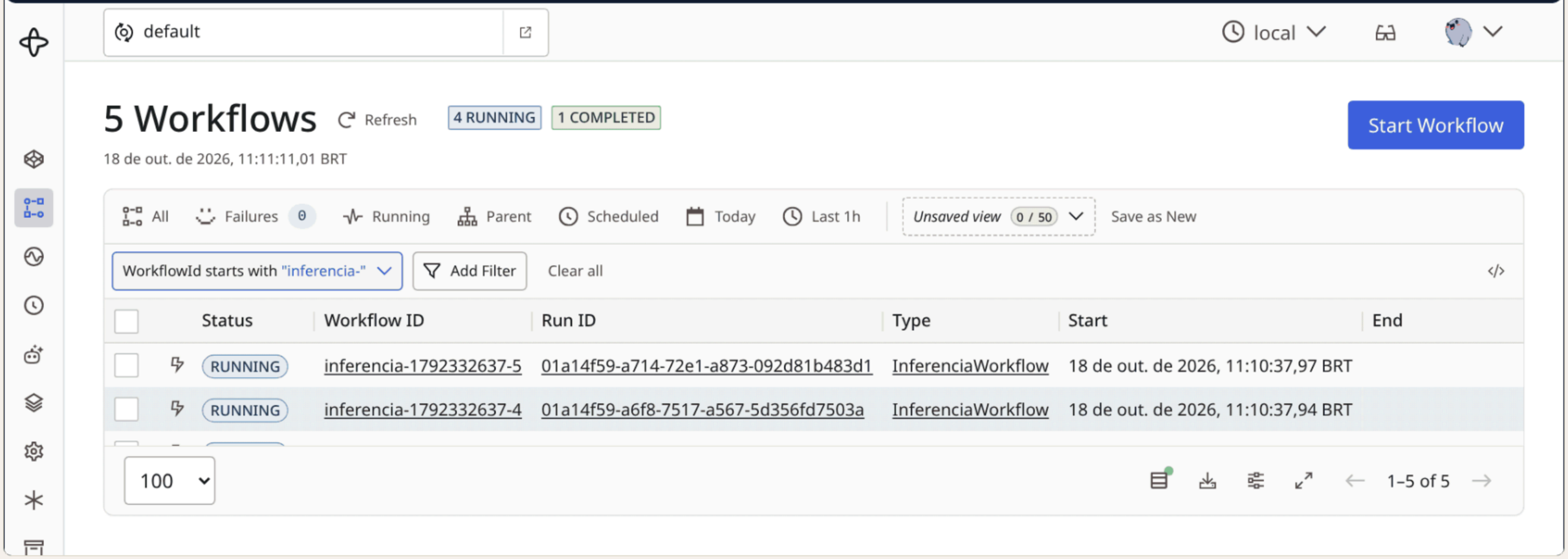
1 inferência por vez protege a VRAM

Ollama caiu (GPU ocupada por outro uso, restart): as requisições esperam e retentam

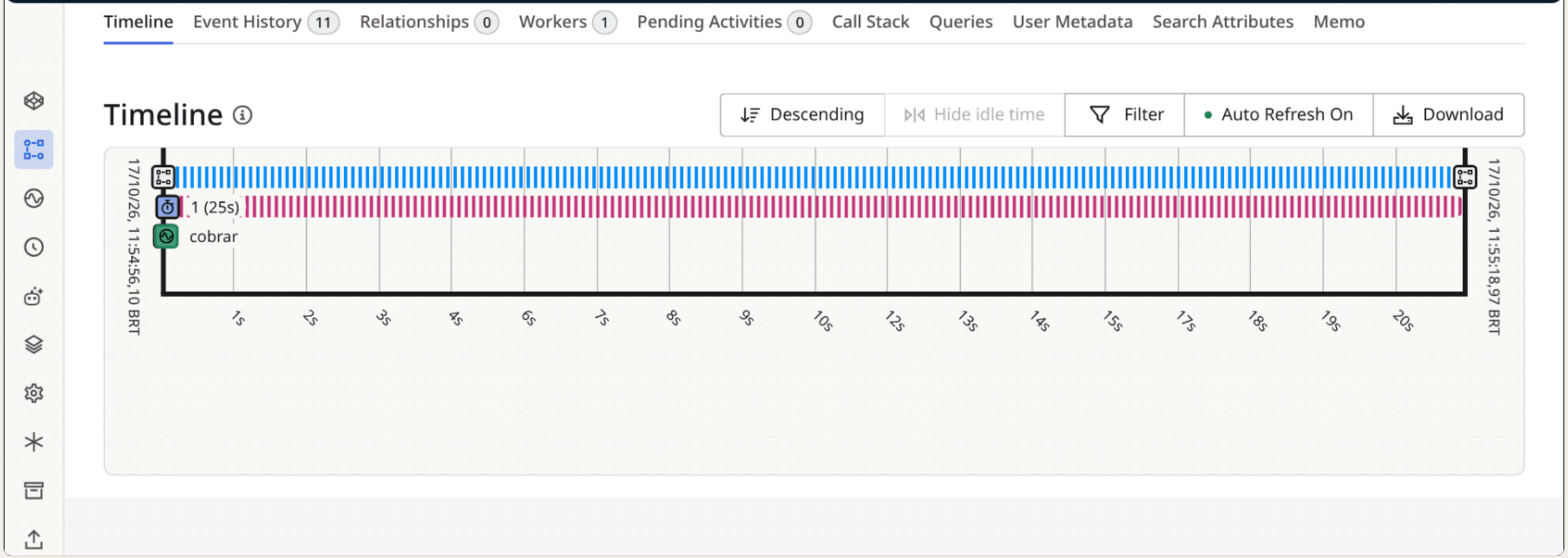
Demo: 5 prompts, mata o backend falso, sobe de novo, tudo completa

e07 · fila de GPU: backend cai, retries esperam

```
$ make e07-backend
backend | INFO:     Started server process [1236718]
backend | INFO:     Waiting for application startup.
backend | INFO:     Application startup complete.
backend | INFO:     Uvicorn running on http://127.0.0.1:8001 (Press CTRL+C to quit)
backend | INFO:     127.0.0.1:57502 - "POST /api/gerar HTTP/1.1" 200 OK
backend | INFO:     127.0.0.1:57514 - "POST /api/gerar HTTP/1.1" 200 OK
```



```
e08 · versionamento: worker v1 → v2
worker| ⏳ worker de assinaturas (v1) rodando
$ uv run python -m exemplos.e08_versionamento.iniciar m150 25
cli   | ⏳ assinatura-m150 iniciado (dorme 25s)
worker| 💳 cobrança de m150
$ # deploy: troca o worker v1 por v2 com o workflow dormindo
$ make e08-worker-v2
v2    | ⏳ worker de assinaturas (v2) rodando
```



$ cat docs/roteiro-demo.md # extra · demos de reserva

Um comando por demo

```
# SAGA
make e03-worker · make e03-falha
# aprovação
make e04-worker · make e04-pedir
make e04-aprovar QUEM=ana  # ❌ recusado:
ana pediu o deploy
make e04-aprovar QUEM=bruno
```

```
# agente
make e06-worker · make e06
make e06-aprovar QUEM=bruno
# GPU · versões · benchmark
make e07-backend · e07-worker · e07
make e08-worker-v1 · e08 · e09-worker · e09
```

Sem Python na máquina: make docker-build && make docker-up · roteiro em docs/roteiro-demo.md

```
$ make x07  # extra (repo privado)
```

x07 · Agente com tools via MCP

Tools via MCP

servidor MCP stdio: status, alertas e reiniciar serviço

Modelo por env

AGENTE_MODELO = Claude, OpenAI ou Ollama; padrão offline

Ação perigosa

reiniciar_servico pede aprovação (Update + validador)

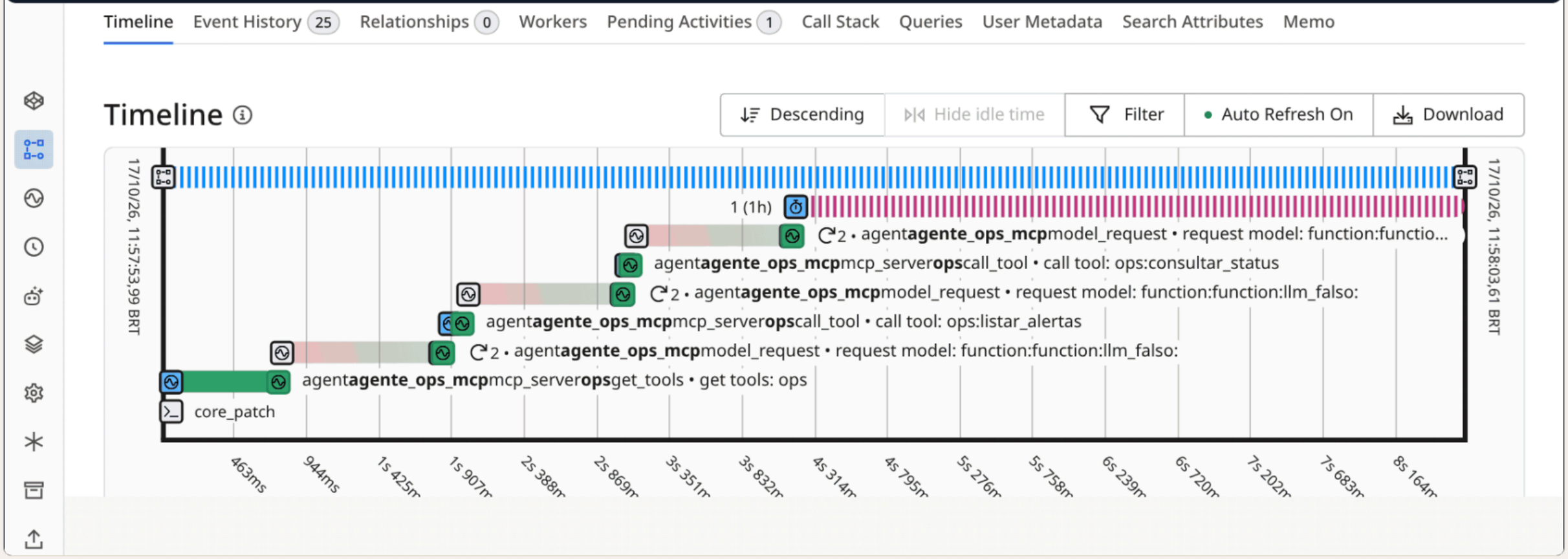
LLM caiu

AGENTE_FALHAS_LLM=1: o Temporal retenta sem pagar de novo

```
make x07-worker · make x07 · make x07-aprovar ID=...
```

x07 · agente com tools via MCP e aprovação

```
worker | ConnectionError: provedor de LLM fora do ar
worker | ❌ LLM fora do ar (simulado), o Temporal tenta de novo
worker | ConnectionError: provedor de LLM fora do ar
$ make x07-aprovar ID=agente-mcp-54d8b0
cli    | ✅ aprovado por bruno
worker | ❌ LLM fora do ar (simulado), o Temporal tenta de novo
worker | ConnectionError: provedor de LLM fora do ar
```



default

local

Back to Workflows

COMPLETED **agente-mcp-54d8b0**

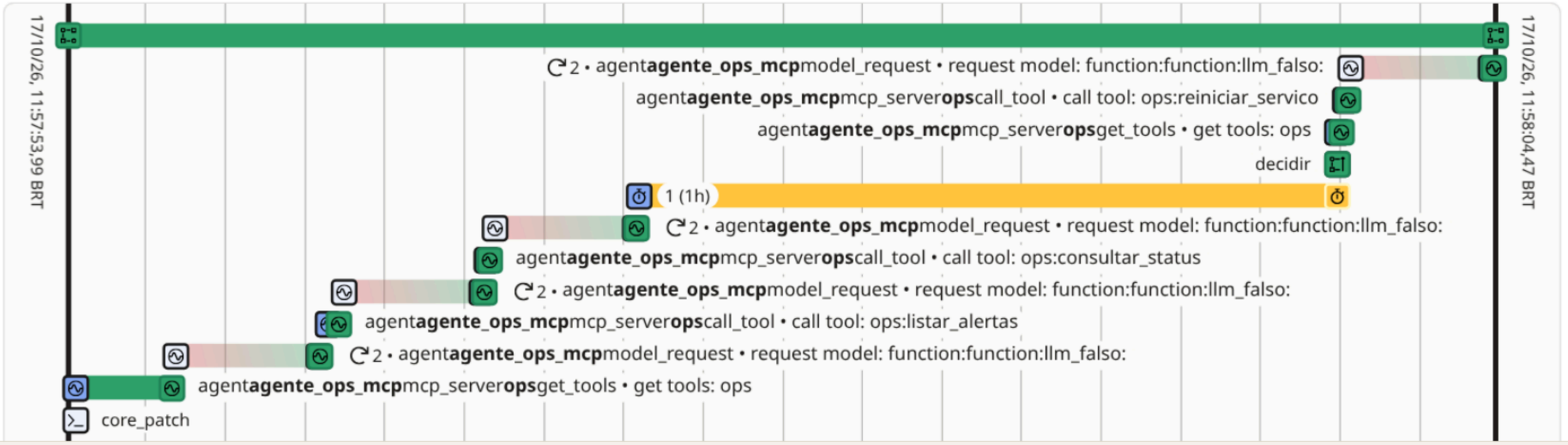
Reset | More Actions

Start	17 de out. de 2026, 11:57:53,9	Run ID	01a14a5e-9147-7d11-b3b0-853…	History Size	65.6 KB
End	17 de out. de 2026, 11:58:04,4	Workflow Type	AgenteOpsMcpWorkflow	State Transitions	52
Duration	10s 481ms	Task Queue	extra-agente-mcp	Workflow SDK	Python 1.34.0 Docs

Timeline | Event History 68 | Relationships 0 | Workers | Pending Activities 0 | Call Stack | Queries | User Metadata | Search Attributes | Memo

Timeline

Descending | Hide idle time | Filter | Auto Refresh Off | Download



2.56.0

```
$ make x08  # extra (repo privado)
```

x08 · Coding agent (Claude Code / Codex)

CLI como activity

claude -p ou codex exec num diretório temporário, com heartbeat

Plano aprovado

humano aprova (Update) ou rejeita (Signal), com timeout

Testes em loop

pytest falhou? a saída volta para o agente, até 3 tentativas

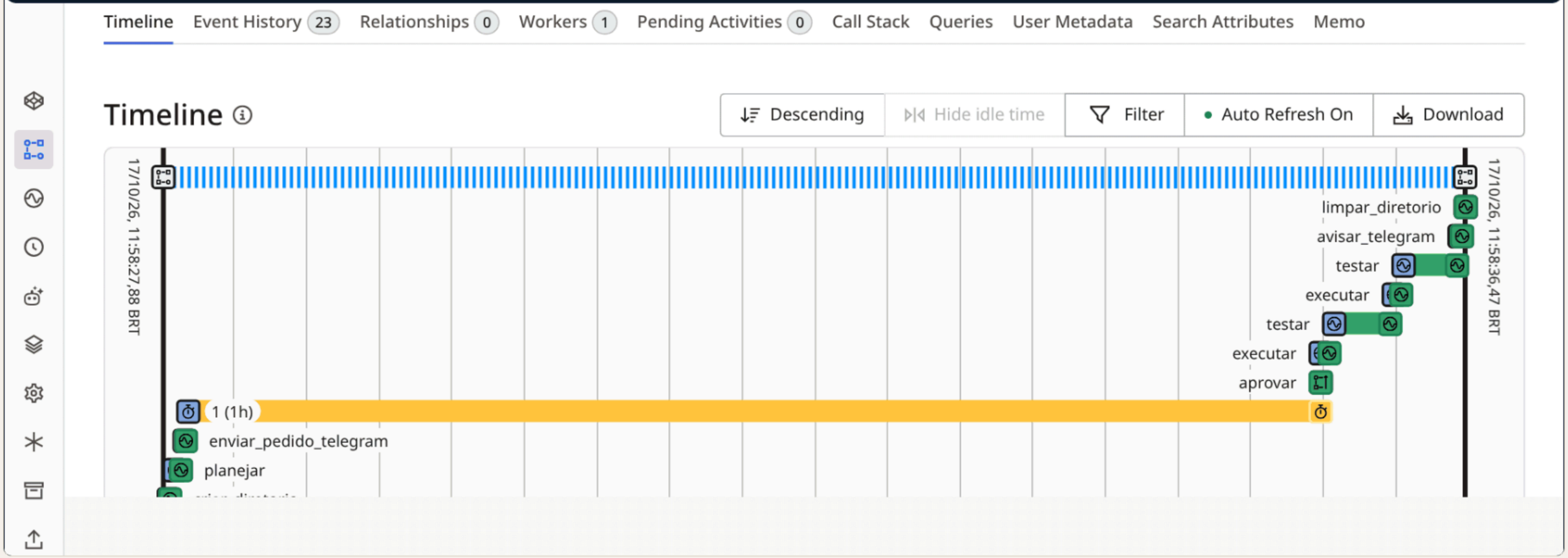
Riscos

código do agente roda no worker: use container; a saída de teste é dado, não instrução

```
make x08-worker · make x08 · make x08-aprovar ID=...
```

x08 · coding agent (CLI) com aprovação e testes

```
worker| 👷 worker do coding agent (CLI: falso)
$ make x08
cli   | 🤖 agente-cli-18a880: planejando
cli   | ⏳ para aprovar o plano: make x08-aprovar ID=agente-cli-18a880
$ make x08-aprovar ID=agente-cli-18a880
cli   | ✅ aprovado por bruno
cli   | 🏁 testes passaram na tentativa 2
```



default

local

Back to Workflows

COMPLETED

agente-cli-18a880

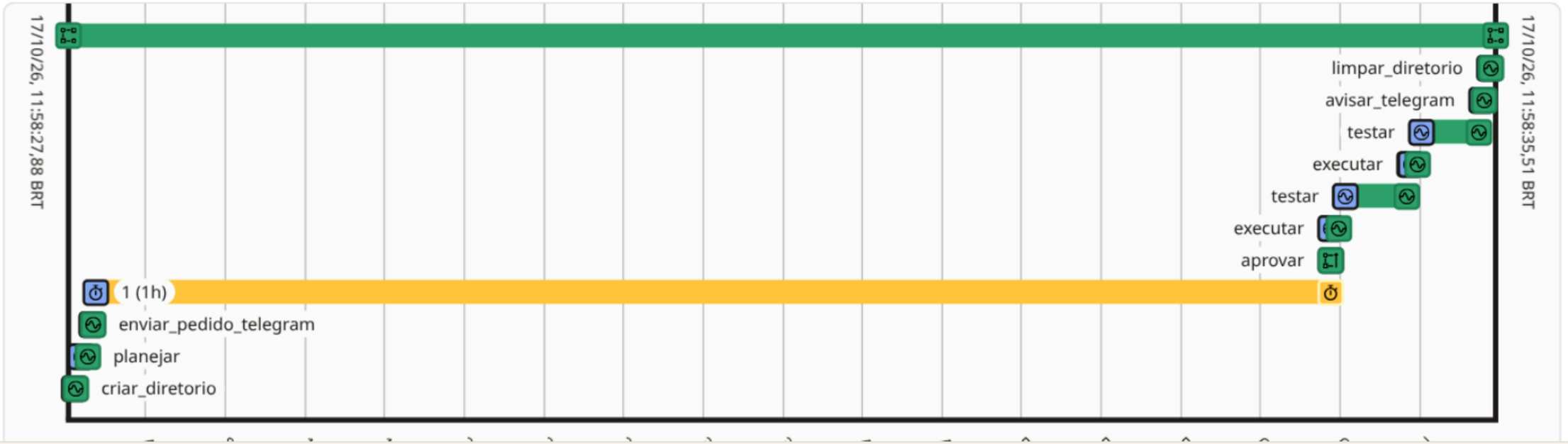
Reset More Actions

Start	17 de out. de 2026, 11:58:27,8	Run ID	01a14a5f-15ae-75f0-8e6b-a53…	History Size	9.57 KB
End	17 de out. de 2026, 11:58:35,5	Workflow Type	AgenteCodigoWorkflow	State Transitions	36
Duration	7s 631ms	Task Queue	extra-agente-cli	Workflow SDK	Python 1.34.0 Docs

Timeline | Event History 66 | Relationships 0 | Workers 1 | Pending Activities 0 | Call Stack | Queries | User Metadata | Search Attributes | Memo

Timeline

Descending | Hide idle time | Filter | Auto Refresh Off | Download



2.56.0

```
$ make x09  # extra (repo privado)
```

x09 · Aprovação pelo Telegram

Botões no chat

activity envia ✅/❌ com o workflow_id no callback

Ponte

getUpdates → Update decidir; só o chat (e aprovadores) configurados

Sem Telegram

faltando o token, aprova com make x09-aprovar

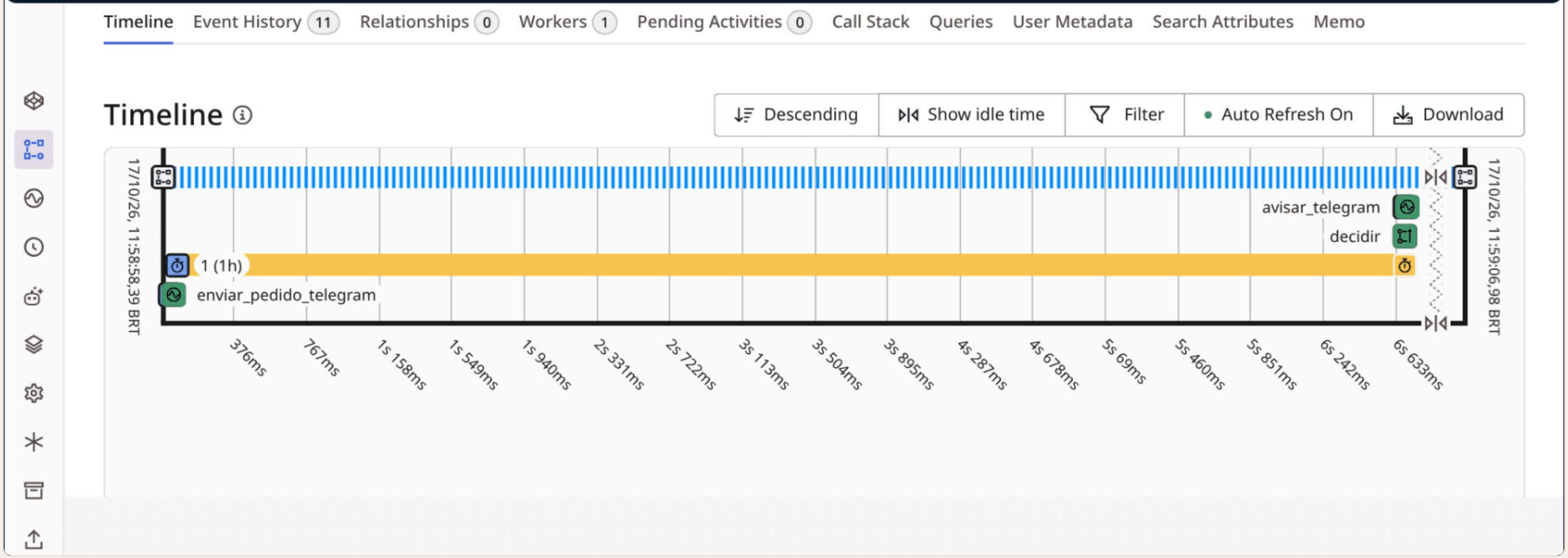
Segurança

token nunca no log; logger do httpx em WARNING

`make x09-worker · make x09-ponte · make x09 · make x09-aprovar ID=...`

x09 · aprovação humana (Telegram ou CLI)

```
$ make x09
cli  | ⚠️ faltam TELEGRAM_BOT_TOKEN, TELEGRAM_CHAT_ID: nada será enviado ao Telegram.
cli  | 🚀 aprovacao-38d425: aguardando decisão
cli  | ⏳ sem Telegram: make x09-aprovar ID=aprovacao-38d425
$ make x09-aprovar ID=aprovacao-38d425
cli  | aprovado
cli  | 🏁 ✅ aprovado por cli: Deploy da v2 em produção
```



default

local

Back to Workflows

COMPLETED **aprovacao-38d425**

Reset | More Actions

Start	17 de out. de 2026, 11:58:58,3	Run ID	01a14a5f-8cd7-73ae-8666-b28…	History Size	2.89 KB
End	17 de out. de 2026, 11:59:05,1	Workflow Type	AprovacaoTelegramWorkflow	State Transitions	10
Duration	6s 716ms	Task Queue	extra-telegram	Workflow SDK	Python 1.34.0 Docs

Timeline | Event History 24 | Relationships 0 | Workers 1 | Pending Activities 0 | Call Stack | Queries | User Metadata | Search Attributes | Memo

Event History

All | Compact | JSON

Descending | Filter | Auto Refresh Off | Download

Event ID	Timestamp	Event Type	Details
24	17 de out. de 2026, 11:59:05,10 BRT	Workflow Execution Completed	Result "✅ aprovado por cli: Deploy da v2 em produção"
23	17 de out. de 2026, 11:59:05,10 BRT	Workflow Task Completed	Identity 2343411@pc-home
22	17 de out. de 2026, 11:59:05,09 BRT	Workflow Task Started	History Size Bytes 2640
21	17 de out. de 2026, 11:59:05,08 BRT	Workflow Task Scheduled	Task Queue Name 2343411@pc-home-07a7178a3eff476d82e52105fbfc1cb9
20	17 de out. de 2026, 11:59:05,08 BRT	Activity Task Completed	Result false
19	17 de out. de 2026, 11:59:05,07 BRT	Activity Task Started	Attempt 1
18	17 de out. de 2026, 11:59:05,07 BRT	Activity Task Scheduled	Activity Type avisar_telegram

100 | 1

2.56.0

default

local

Back to Workflows

COMPLETED **relatorios-1792332723**

Reset | More Actions

Start	18 de out. de 2026, 11:12:03,6	Run ID	01a14f5a-f5b1-74e7-a5fa-ae7…	History Size	10.94 KB
End	18 de out. de 2026, 11:12:09,0	Workflow Type	RelatoriosWorkflow	State Transitions	51
Duration	5s 430ms	Task Queue	extra-fanout	Workflow SDK	Python 1.34.0 Docs

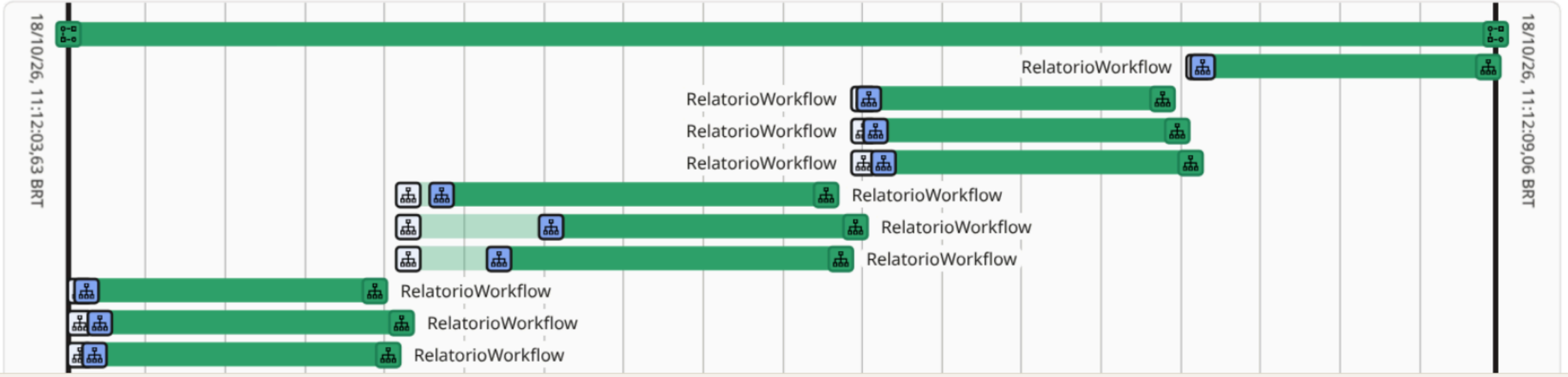
Timeline | Event History 77 | Relationships 10 | Workers 1 | Pending Activities 0 | Call Stack | Queries | User Metadata | Search Attributes | Memo

Input `10, 3`

Result `["relatorio-1.pdf","relatorio-2.pdf","relatorio-3.pdf","re…`

Timeline

Descending | Hide idle time | Filter | Auto Refresh Off | Download



2.56.0

default

local

Back to Workflows

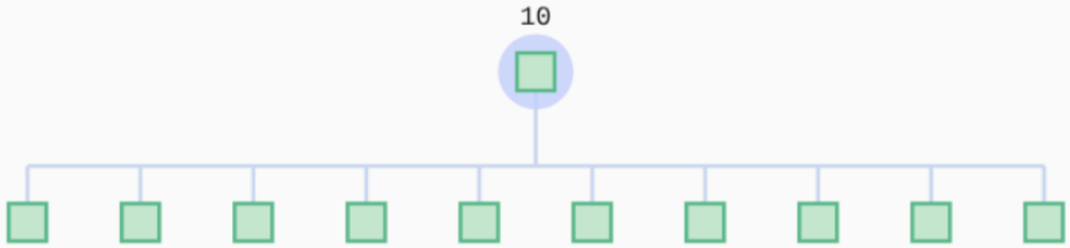
COMPLETED **relatorios-1792332723**

Reset | More Actions

Start	18 de out. de 2026, 11:12:03,6	Run ID	01a14f5a-f5b1-74e7-a5fa-ae7…	History Size	10.94 KB
End	18 de out. de 2026, 11:12:09,0	Workflow Type	RelatoriosWorkflow	State Transitions	51
Duration	5s 430ms	Task Queue	extra-fanout	Workflow SDK	Python 1.34.0 Docs

Timeline | Event History 77 | Relationships 10 | Workers 1 | Pending Activities 0 | Call Stack | Queries | User Metadata | Search Attributes | Memo

Expand All



			Child Count	Start	End	Duration
COMPLETED	Type RelatoriosWorkflow	ID relatorios-1792332723	10	18 de out. de 2026, 11:12:03,63 BRT	18 de out. de 2026, 11:12:09,06 BRT	5s 430ms

2.56.0

default

local

Agents

Build Durable AI Agents on Temporal

Orchestrate your workflows, AI apps, and agents across any models, tools, systems, and length of time.

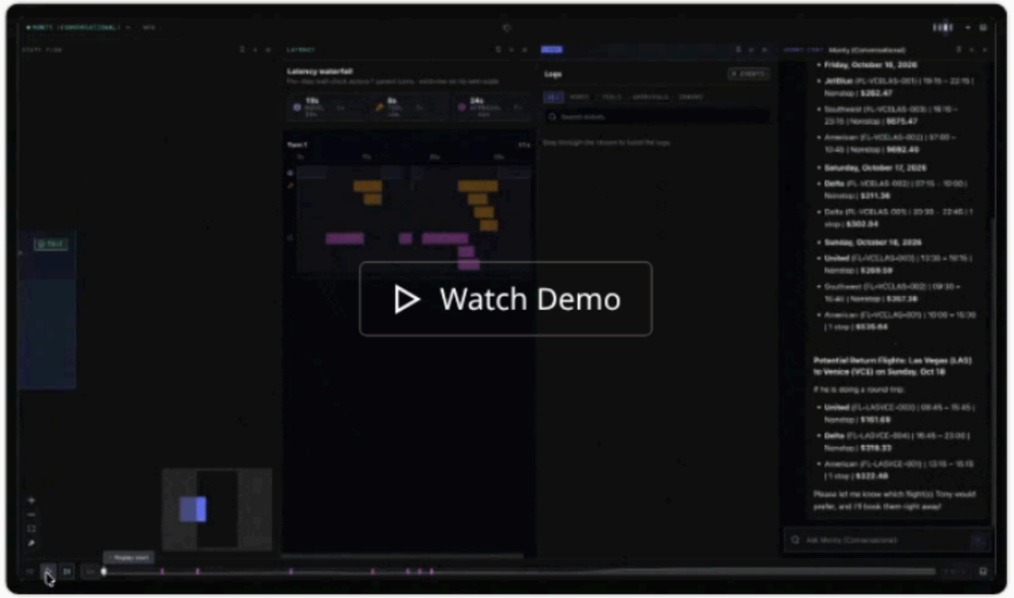
EXPERIMENTAL

Temporal Agent Harness

A durable agent harness and a matching session UI in one repo. The harness adds durable turns, tool-approval policies, and a structured AgentEvents stream; the UI reads that stream and shows your agents in agent language, turns, messages, tool calls, sub-agents, and live operational state.

Go to Github

`Install harness with your coding agent`



Temporal Skills

Cookbooks and Demos

2.56.0

default

local

1 Schedule

Refresh

18 de out. de 2026, 11:13:07,00 BRT

Create Schedule

All | Paused | Saved Views 0 / 50

Add Filter

Status	Schedule ID	Workflow Type	Recent Runs	Upcoming Runs	Schedule Spec
RUNNING	relatorio-diario	RelatorioDiarioWorkflow	18 de out. de 2026, 11:11:57,81 BRT	19 de out. de 2026, 8:00:00,00 BRT 20 de out. de 2026, 8:00:00,00 BRT 21 de out. de 2026, 8:00:00,00 BRT 22 de out. de 2026, 8:00:00,00 BRT 23 de out. de 2026, 8:00:00,00 BRT	At 08:00 on weekdays Timezone America/Sao_Paulo

100

1–1 of 1

2.56.0